



StreamTeX Training Course: Advanced

A Streamlit-based content rendering framework

ADVANCED LEVEL

Master Advanced Patterns and Enterprise Features

This course covers advanced StreamTeX patterns for production use. Multi-source blocks, collections, export, and deployment strategies.

- **Registry:** LazyBlockRegistry, shared blocks, static resolution
- **Advanced:** overlays, themes, helpers, collections
- **Production:** export, deployment, data visualization

Multi-source lazy-loading for enterprise-scale block management

What is LazyBlockRegistry?

Purpose

Key idea: Instead of duplicating blocks across projects, place shared blocks

in a common directory and point LazyBlockRegistry at it.

Each project can combine its own local blocks with shared ones.

The registry is defined in `streamtex/blocks.py` and exposed via the top-level `streamtex` namespace as `stx.LazyBlockRegistry`.

Creating a Registry

Purpose

```
1 import streamtex as stx
2 from pathlib import Path
3
4 # Point to a shared blocks directory
5 shared_path = str(Path(__file__).parent.parent / "shared-blocks" / "blocks")
6
```

wrap

```
7
8 # Create the registry
shared_blocks = stx.LazyBlockRegistry([shared_path])
```

Details:

Accessing Blocks via Attribute Access

Purpose

```
1 # Access a block – triggers lazy import on first access
2 module = shared_blocks.bck_header_training
3
4 # The module has a build() function
5 module.build() # Renders the block content
6
7 # Use directly in st_book() – st_book calls build() for you
8 from streamtex import st_book
9 st_book([
10     shared_blocks.bck_header_training, # Lazy-loaded here
11     blocks.bck_content_01,             # Local block
12     shared_blocks.bck_footer_training, # Lazy-loaded here
13 ])
```

How `__getattr__` works: When you write `shared_blocks.bck_name`, Python calls the registry's `__getattr__` method. It searches each source directory

for a file named `bck_name.py`, imports it with `importlib.util`, caches the module, and returns it.

Details:

Priority Order (First Source Wins)

Purpose

```
1  import streamtex as stx
2
3  # Priority: project_overrides > shared_blocks > fallback_blocks
4  registry = stx.LazyBlockRegistry([
5      "blocks/overrides",      # Highest priority (checked first)
6      "../../shared/blocks",   # Medium priority
7      "../../fallback/blocks", # Lowest priority (checked last)
8  ])
9
10 # If bck_header.py exists in both "overrides" and "shared",
11 # the version from "overrides" is used.
12 registry.bck_header # -> loads from blocks/overrides/bck_header.py
```

Use case: A university has standard course headers/footers in a shared directory. Each professor can override specific blocks in their project's local override directory without modifying the shared originals.

Caching Behavior

Purpose

```
# Internal cache structure:
1 # registry._cache = {}          # block_name -> module
2 # registry._not_found = set()   # block names already searched (not found)
3
4 # First access: filesystem search + import + cache
5 module = registry.bck_welcome  # Slow (disk I/O + import)
6
7
8 # Second access: instant cache hit
9 module = registry.bck_welcome  # Fast (dict lookup)
10
11 # Not-found blocks are also cached to avoid repeated searches
12 try:
13     registry.bck_nonexistent    # Searches all directories once
14 except AttributeError:
15     pass
16
17 # Subsequent attempts skip the search entirely
18 try:
19     registry.bck_nonexistent    # Instant AttributeError (cached miss)
20 except AttributeError:
21     pass
```

[wrap](#)

Details:

LazyBlockRegistry vs ProjectBlockRegistry

Purpose

ProjectBlockRegistry (Single-Project)

Purpose: Manage blocks within a single project's blocks/ directory.

Features: Manifest-based discovery, block type detection (atomic vs composite), introspection (list_blocks, get_stats, load_all).

Usage: Typically used in blocks/__init__.py to auto-discover all bck_*.py files.

```
1  # blocks/__init__.py – ProjectBlockRegistry for local blocks
2  from pathlib import Path
3  from streamtex import ProjectBlockRegistry
4
5  registry = ProjectBlockRegistry(Path(__file__).parent)
6
7  def __getattr__(name):
8      return registry.get(name)
9
10 # Usage in book.py:
11 import blocks
12 blocks.bck_welcome      # Auto-discovered from blocks/bck_welcome.py
13 blocks.bck_content_01   # Auto-discovered from blocks/bck_content_01.py
```

LazyBlockRegistry (Multi-Source)

Purpose: Load blocks from external directories (shared blocks, other projects, override folders).

Features: Multi-source with priority, lazy-loading, caching, attribute access.

Usage: Created in book.py to reference shared blocks alongside local ones.

When to Use Which?

Details:

Real-World Example: This Project's book.py

Purpose

```
1  # book.py — stx_manual_advanced
2
3  import streamlit as st
4  import setup
5  import streamtex as stx
6  from streamtex import st_book, TOCConfig, MarkerConfig
7  from pathlib import Path
8
9
10 import blocks  # Local blocks (ProjectBlockRegistry in __init__.py)
11
12 # Configure shared blocks (LazyBlockRegistry)
13 _shared_blocks_path = str(
14     Path(__file__).parent.parent / "shared-blocks" / "blocks"
```

wrap

```

15 )
16 shared_blocks = stx.LazyBlockRegistry([_shared_blocks_path])
17
18 # Configure static sources for multi-directory image resolution
19 stx.set_static_sources([
20     str(Path(__file__).parent / "static"),          # Local static first
21     str(Path(__file__).parent.parent / "shared-blocks" / "static"),
22 ])
23
24 # Orchestrate: mix local and shared blocks
25 st_book([
26     blocks.bck_level_badge,                          # Local block
27     blocks.bck_lazy_block_registry_demo,             # Local block (this page!)
28     blocks.bck_shared_blocks_usage,                  # Local block
29     # ... more local blocks ...
30     shared_blocks.bck_footer_training,               # Shared block (lazy-loaded)
31 ], paginate=True)

```

Key pattern: Local blocks use 'blocks.bck_name' (via ProjectBlockRegistry).
 Shared blocks use 'shared_blocks.bck_name' (via LazyBlockRegistry).
 Both appear in the same st_book() call seamlessly.

The shared-blocks/ Directory Structure

```

1 documentation/manuals/
2   shared-blocks/
3     blocks/
4       bck_header_training.py    # Shared header for training courses
5       bck_footer_training.py    # Shared footer for training courses
6       bck_welcome.py            # Shared welcome page
7       bck_style_basics.py       # Shared style introduction
8       bck_best_practices.py     # Shared best practices
9

```

```
10     static/  
11     images/                # Shared images  
12     stx_manual_intro/      # Uses shared blocks via LazyBlockRegistry  
    stx_manual_advanced/     # Uses shared blocks via LazyBlockRegistry
```

Details:

Best Practices

Purpose

1. Keep shared blocks self-contained

Shared blocks should not depend on project-specific `custom/styles.py`.
Use `StxStyles` directly or accept styles as parameters.

2. Use `set_static_sources()` for shared images

When shared blocks reference images, configure `stx.set_static_sources()`
in `book.py` so the image resolver can find assets across directories.

3. Place overrides in a dedicated directory

If you need to customize a shared block for one project,
create an `overrides/` folder and list it first in the `sources` array.

4. Use `repr()` for debugging

Call `repr(registry)` or `print(registry)` to see the current state:
source paths and number of cached blocks.

```
1  import streamtex as stx
2
3  registry = stx.LazyBlockRegistry(["shared/blocks", "fallback/blocks"])
4  print(registry)
5  # -> LazyBlockRegistry(sources=['/abs/shared/blocks', '/abs/fallback/blocks'], cached=0)
6
7  _ = registry.bck_header
8  print(registry)
9  # -> LazyBlockRegistry(sources=[...], cached=1)
```

Reusable block libraries for multi-project StreamTeX architectures

What Are Shared Blocks?

Purpose

Why share blocks? Training courses, corporate templates, and multi-project sites often need identical headers, footers, style guides, or welcome pages. Instead of duplicating these across projects, place them in a shared directory and load them with LazyBlockRegistry.

Key principle: A shared block is structurally identical to a local block. It has a BlockStyles class, a build() function, and follows all the same coding standards. The only difference is where it lives on disk and how it is loaded.

Directory Structure for Shared Blocks

Purpose

```
1 # Shared block library directory structure
2 shared-blocks/
3   blocks/
4     __init__.py           # Module init (imports or discovery)
5     bck_header_training.py # Shared header block
6     bck_footer_training.py # Shared footer block
7     bck_welcome.py        # Shared welcome page
8     bck_style_basics.py   # Shared style introduction
9     bck_best_practices.py  # Shared best practices guide
10  custom/
11    styles.py              # SharedStyles class (inherits StxStyles)
12  static/
13    images/                # Shared image assets
14      logo.png
15      banner.png
```

Details:

How This Test Suite Is Organized

```
1 # In this repository, shared blocks sit at documentation/manuals/shared-blocks/
2 documentation/manuals/
3   shared-blocks/  # Shared block library
4     blocks/
5       bck_header_training.py
6       bck_footer_training.py
7       bck_welcome.py
8       bck_style_basics.py
9       bck_best_practices.py
10  custom/
11    styles.py          # SharedStyles(StxStyles)
```

```
12     static/
13     images/
14     stx_manual_intro/           # Project A — uses shared blocks
15     book.py
16     blocks/
17     custom/
18     static/
19     stx_manual_advanced/       # Project B — also uses shared blocks
20     book.py
21     blocks/
22     custom/
23     static/
```

Loading Shared Blocks with LazyBlockRegistry

Purpose

```
1  # book.py — Loading shared blocks
2  import streamtex as stx
3  from pathlib import Path
4
5  # Resolve the path to the shared blocks directory
6  _shared_blocks_path = str(
7      Path(__file__).parent.parent / "shared-blocks" / "blocks"
8  )
9
10
11  # Create a LazyBlockRegistry for shared blocks
```

```
12 shared_blocks = stx.LazyBlockRegistry([_shared_blocks_path])
13
14 # Now access any shared block by name (dot notation)
15 # The first access triggers a lazy import; subsequent accesses are cached
16 shared_blocks.bck_header_training    # -> loads shared-blocks/blocks/bck_header_training.py
    shared_blocks.bck_footer_training  # -> loads shared-blocks/blocks/bck_footer_training.py
```

How it works internally: When you write `shared_blocks.bck_name`, Python calls `__getattr__` on the registry. It searches the source directories for `bck_name.py`, imports it with `importlib.util.spec_from_file_location`, caches the module, and returns it. If not found, an `AttributeError` is raised.

Details:

Using Shared Blocks Alongside Local Blocks

Purpose

```
1 # book.py — Combining local and shared blocks in st_book()
2 import streamlit as st
3 import setup
4 import streamtex as stx
5 from streamtex import st_book, TOCConfig, MarkerConfig
6 from pathlib import Path
7
8 import blocks # Local blocks (ProjectBlockRegistry in __init__.py)
9
10 # Shared blocks via LazyBlockRegistry
11
```

```

12 _shared_path = str(Path(__file__).parent.parent / "shared-blocks" / "blocks")
13 shared_blocks = stx.LazyBlockRegistry([_shared_path])
14
15 # Orchestrate: mix local and shared blocks freely
16 st_book([
17     shared_blocks.bck_header_training,      # Shared header (from shared-blocks/)
18     blocks.bck_welcome_intro,               # Local block (from blocks/)
19     blocks.bck_text_and_styling,            # Local block
20     blocks.bck_grids_and_lists,             # Local block
21     shared_blocks.bck_best_practices,       # Shared block
22     shared_blocks.bck_footer_training,      # Shared footer (from shared-blocks/)
23 ], paginate=True)

```

Pattern: Use 'blocks.bck_name' for local blocks and 'shared_blocks.bck_name' for shared blocks. The prefix makes it immediately clear where each block comes from, improving code readability and maintainability.

This Project's Actual book.py

Purpose

```

1  # From documentation/manuals/stx_manual_advanced/book.py (actual code)
2
3  import blocks  # Local blocks via ProjectBlockRegistry
4
5  # Configure shared blocks
6  _shared_blocks_path = str(

```

wrap

```

7     Path(__file__).parent.parent / "shared-blocks" / "blocks"
8 )
9 shared_blocks = stx.LazyBlockRegistry([_shared_blocks_path])
10
11 st_book([
12     blocks.bck_level_badge,                # Local
13     blocks.bck_lazy_block_registry_demo,    # Local
14     blocks.bck_shared_blocks_usage,         # Local (this page!)
15     blocks.bck_static_resolution_demo,      # Local
16     # ... more local blocks ...
17     shared_blocks.bck_footer_training,      # Shared (lazy-loaded)
18 ], paginate=True)

```

The Shared custom/styles.py Pattern

Purpose

```

1  # shared-blocks/custom/styles.py
2  from streamtex.styles import StxStyles, Style
3
4
5  class SharedStyles(StxStyles):
6      """Shared style palette for training courses."""
7
8      class colors:
9          """Shared color palette"""
10
11         primary_blue = Style("color: #667eea;", "shared_primary_blue")
12         accent_teal = Style("color: #20B2AA;", "shared_accent_teal")
13         warning_orange = Style("color: #FF8C00;", "shared_warning_orange")

```

```

14
15     class titles:
16         """Shared title styles"""
17         course_title = StxStyles.GIANT + colors.primary_blue
18         section_title = StxStyles.Large + colors.primary_blue
19         subsection_title = StxStyles.large + colors.accent_teal

```

Key design choice: SharedStyles inherits from StxStyles, not from any project's Styles class. This makes shared blocks portable: they work in any project regardless of that project's custom style definitions.

Tip: Shared blocks should use StxStyles (stx.StxStyles) or their own SharedStyles for styling. They should never import from 'custom.styles import Styles as s' because that path resolves to the consuming project's styles, not the shared library's styles. This coupling would break portability.

How Shared Blocks Use SharedStyles

```

1  # shared-blocks/blocks/bck_header_training.py
2  import streamtex as stx
3  from streamtex import st_write, st_block, st_space
4  from streamtex.styles import Style
5  from streamtex.enums import Tags as t
6
7
8  class BlockStyles:
9      """Shared header styles."""
10
11      header_bg = Style(
12          "background:linear-gradient(135deg, #667eea 0%, #764ba2 100%);"
13          "padding:40px 20px;border-radius:8px;",
14          "training_header"
15      )

```

```
16
17
18 def build():
19     """Render a standard training course header."""
20     st_space("v", 1)
21     with st_block(BlockStyles.header_bg):
22         st_write(
23             stx.StxStyles.LARGE + stx.StxStyles.text.colors.white,
24             "StreamTeX Training Course",
25             tag=t.div
26         )
27         st_write(
28             stx.StxStyles.large + stx.StxStyles.text.colors.white,
29             "A Streamlit-based content rendering framework",
30             tag=t.div
31         )
32     st_space("v", 1)
```

Details:

Static Assets in Shared Blocks

Purpose

```
1 # book.py – Configure static sources for multi-directory resolution
2 import streamtex as stx
3 from pathlib import Path
4
```

wrap

```

5 # Local static directory (highest priority)
6 _local_static = str(Path(__file__).parent / "static")
7
8 # Shared static directory (fallback)
9 _shared_static = str(
10     Path(__file__).parent.parent / "shared-blocks" / "static"
11 )
12
13 # Register both sources: local first, shared second
14 stx.set_static_sources([
15     _local_static,      # Checked first (project-specific assets)
16     _shared_static,     # Checked second (shared assets)
17 ])

```

Resolution order: When a block calls `st_image(uri='images/logo.png')`, the image resolver searches the static sources in order. If `logo.png` exists in the project's local `static/images/`, that version is used. Otherwise, the shared `static/images/logo.png` is used. This lets projects override shared assets when needed.

```

1 # Shared block referencing a shared image
2 from streamtex import st_image
3
4 def build():
5     # This image will be resolved from shared-blocks/static/images/
6     # unless the consuming project has its own static/images/logo.png
7     st_image(uri="images/logo.png")

```

Details:

Creating Your Own Shared Block Library

Purpose

Step 1: Create the Directory Structure

```
1 # Create the shared library directory
2 mkdir -p my-shared-blocks/blocks
3 mkdir -p my-shared-blocks/custom
4 mkdir -p my-shared-blocks/static/images
5
6 # Create the __init__.py for the blocks package
7 touch my-shared-blocks/blocks/__init__.py
```

Step 2: Write the Shared Styles

```
1 # my-shared-blocks/custom/styles.py
2 from streamtex.styles import StxStyles, Style
3
4
5 class SharedStyles(StxStyles):
6     """Organization-wide style palette."""
7
8     class colors:
9         brand_primary = Style("color: #1A73E8;", "brand_primary")
```

```

10         brand_accent = Style("color: #34A853;", "brand_accent")
11
12     class titles:
13         page_title = StxStyles.LARGE + colors.brand_primary

```

Step 3: Write a Shared Block

```

1  # my-shared-blocks/blocks/bck_org_header.py
2  """Organization-wide header block."""
3
4  import streamtex as stx
5  from streamtex import st_write, st_block, st_space
6  from streamtex.styles import Style
7  from streamtex.enums import Tags as t
8
9
10
11 class BlockStyles:
12     """Organization header styles."""
13     banner = Style(
14         "background:linear-gradient(135deg, #1A73E8 0%, #34A853 100%);"
15         "padding:32px 24px;border-radius:12px;",
16         "org_banner"
17     )
18
19 bs = BlockStyles
20
21
22 def build():
23     """Render the organization header."""
24     with st_block(bs.banner):
25         st_write(
26             stx.StxStyles.LARGE + stx.StxStyles.text.colors.white,
27             "My Organization",

```

```

28         tag=t.div
29     )
30     st_write(
31         stx.StxStyles.large + stx.StxStyles.text.colors.white,
32         "Powered by StreamTeX",
33         tag=t.div
34     )
    st_space("v", 2)

```

Step 4: Consume from a Project

```

1  # my-project/book.py
2  import streamlit as st
3  import setup
4  import streamtex as stx
5  from streamtex import st_book
6  from pathlib import Path
7
8  import blocks  # Local blocks
9
10 # Load shared blocks
11 _shared_path = str(
12     Path(__file__).parent.parent / "my-shared-blocks" / "blocks"
13 )
14 )
15 shared = stx.LazyBlockRegistry([_shared_path])
16
17 # Configure shared static assets
18 stx.set_static_sources([
19     str(Path(__file__).parent / "static"),
20     str(Path(__file__).parent.parent / "my-shared-blocks" / "static"),
21 ])
22
23 # Build the book with both local and shared blocks

```

```

24 st_book([
25     shared.bck_org_header,      # Shared header
26     blocks.bck_intro,          # Local content
27     blocks.bck_chapter_1,      # Local content
28     blocks.bck_chapter_2,      # Local content
29     shared.bck_org_footer,      # Shared footer
    ], paginate=True)

```

Best Practices

Purpose

1. Keep shared blocks self-contained

A shared block should never import from a project-specific custom/styles.py. Use stx.StxStyles or define styles locally in the block's BlockStyles class. This ensures the block works in any project without modification.

```

1  # GOOD: Self-contained shared block
2  import streamtex as stx
3  from streamtex.styles import Style
4
5  class BlockStyles:
6      title = stx.StxStyles.Large + stx.StxStyles.bold
7      container = Style("padding:16px;border-radius:8px;", "shared_container")
8
9
10 # BAD: Coupled to a specific project
11 from custom.styles import Styles as s # Which project's styles?
12

```

```
class BlockStyles:
    title = s.project.titles.section_title # Breaks in other projects!
```

2. Avoid external dependencies

Shared blocks should depend only on streamtex and streamlit.
If a block needs additional packages, document it clearly or consider whether the block truly belongs in the shared library.

3. Use `set_static_sources()` for images

Every project that consumes shared blocks with images must call `stx.set_static_sources()` in its `book.py`. Include the shared static directory after the local one so project assets take priority.

4. Override shared blocks with priority sources

To customize a shared block for one project, create an `overrides/` directory with the same filename and place it first in the `LazyBlockRegistry` sources list. The first source wins.

```
1 # Override pattern: project-specific version takes priority
2 shared = stx.LazyBlockRegistry([
3     "blocks/overrides", # Project overrides (checked first)
4     "../../shared-blocks/blocks", # Shared originals (checked second)
5 ])
6
7 # If blocks/overrides/bck_header.py exists, it is used
8 # Otherwise, shared-blocks/blocks/bck_header.py is used
9 shared.bck_header # -> resolves to highest-priority source
```

5. Use consistent naming conventions

Prefix shared block filenames to indicate their purpose: `bck_header_*`, `bck_footer_*`, `bck_shared_*`. This makes it clear at a glance which blocks are designed to be shared versus project-specific.

6. Use unique style IDs across projects

Style IDs are global (they generate CSS class names). Prefix shared style IDs with `'shared_'` or your organization name to avoid collisions with project-specific styles.

```
1  # Shared styles with prefixed IDs to avoid collisions
2  Style("color:#667eea;", "shared_primary_blue")      # Prefixed with 'shared_'
3  Style("padding:24px;", "acme_card_container")        # Prefixed with org name
4
5  # Avoid generic IDs that may clash with project styles
6  Style("color:#667eea;", "primary_blue")              # Risk of collision!
```

7. Document your shared library

Include a README in the `shared-blocks/` root that lists all available blocks, their purpose, and any configuration they expect. This helps new projects discover and adopt shared blocks quickly.

Summary: Shared blocks let you build once and reuse everywhere. Use `LazyBlockRegistry` to load them, `set_static_sources()` for images, keep blocks self-contained with no project-specific imports, and leverage priority ordering for project-specific overrides.

How StreamTeX resolves images and static files across multiple directories

What is Static Resolution?

Purpose

Key idea: When multiple projects share static assets (images, data files), static resolution lets each project reference them by name without knowing their exact filesystem location.

The resolution system is defined in `streamtex/blocks.py` and exposed as `stx.set_static_sources()`, `stx.get_static_sources()`, and `stx.resolve_static()`. The `st_image()` function uses it internally for automatic image discovery.

Configuring Static Sources with `set_static_sources()`

Purpose

```
1  import streamtex as stx
2  from pathlib import Path
3
4
```

wrap

```
5 # Configure static sources in book.py (called once at startup)
6 stx.set_static_sources([
7     str(Path(__file__).parent / "static"),          # Local project
8     static
9     str(Path(__file__).parent.parent / "shared-blocks" / "static"), # Shared static
10 ])
```

Order matters: The first directory in the list has the highest priority.
If a file exists in multiple directories, the first match wins.
Always place your local project's static/ directory first.

Details:

resolve_static() — How It Searches Directories

Purpose

```
1 import streamtex as stx
2
3 # Resolve a data file across static sources
4 data_path = stx.resolve_static("data/trainers.json")
5
6 # The function searches:
7 # 1. /abs/path/to/project/static/data/trainers.json -> found? return it
8 # 2. /abs/path/to/shared-blocks/static/data/trainers.json -> found? return it
9 # 3. Not found anywhere -> returns "data/trainers.json" (original path)
10
11 # Use the resolved path
12
```

```
13 import json
14 with open(data_path) as f:
    trainers = json.load(f)
```

The Search Algorithm

```
1 # Simplified internal logic of resolve_static():
2 def resolve_static(relative_path: str) -> str:
3     for base in _static_sources:
4         full_path = os.path.join(base, relative_path)
5         if os.path.exists(full_path):
6             return full_path          # First match wins
7     return relative_path             # Fallback: original path
```

Details:

Priority Order (First Path Wins)

Purpose

```
1 import streamtex as stx
2
3 # Priority: local > shared > fallback
4 stx.set_static_sources([
5     "static",          # Highest priority (project-local)
6     "../..//shared/static", # Medium priority (team-shared)
7     "../..//defaults/static", # Lowest priority (organization defaults)
```

```
8     1)
9
10    # If logo.png exists in ALL three directories:
11    # resolve_static("images/logo.png")
12    # -> returns "/abs/path/to/static/images/logo.png"    (local wins)
13
14    # If logo.png only exists in shared and defaults:
15    # resolve_static("images/logo.png")
16    # -> returns "/abs/path/to/shared/static/images/logo.png"    (shared wins)
```

Override pattern: A university provides default logos and banners in a shared directory. Each department can override specific images by placing a same-named file in their local static/ directory. No configuration changes needed — the priority order handles it automatically.

How `st_image()` Uses Static Resolution Internally

Purpose

The URI Resolution Flow

```
1    # What happens inside st_image(uri="logo.png"):
2
3    # Step 1: Is it a URL? (http://, https://, www.)
4    #     -> No, continue
5
6    # Step 2: Is it an absolute path? (os.path.isabs)
```

```

7  # -> No, continue
8
9  # Step 3: Is it a relative path? (starts with '.', '..', '/')
10 # -> No, continue
11
12 # Step 4: Static resolution — search configured sources
13 for base in get_static_sources():
14     for subdir in ["images", ""]:
15         full_path = os.path.join(base, subdir, uri)
16         if os.path.isfile(full_path):
17             # Found! Convert to base64 and return
18             return encode_to_base64(full_path)
19
20
21 # Step 5: Fallback — use Streamlit's built-in static serving
    return f"app/static/images/{uri}"

```

Automatic subdirectory search: For each static source, `st_image()` checks both the 'images/' subdirectory and the root of the static directory. This means `st_image(uri='logo.png')` will find the file whether it lives at `static/images/logo.png` or `static/logo.png`.

```

# All of these work with static resolution:
1  st_image(uri="logo.png")           # Searches static/images/logo.png, then static/logo.png
2  st_image(uri="icons/arrow.png")    # Searches static/images/icons/arrow.png, then
3  static/icons/arrow.png
4
5  # These bypass static resolution (direct path handling):
6  st_image(uri="https://example.com/logo.png")  # URL — used directly
7  st_image(uri="/abs/path/to/logo.png")         # Absolute path — base64 encoded
8  st_image(uri="./local/logo.png")              # Relative path — resolved from cwd

```

Multi-Directory Patterns

Purpose

Pattern 1: Local + Shared Static

```

# The most common pattern: project-local + shared static directories
1 # Used in stx_manual_advanced/book.py
2
3 stx.set_static_sources([
4     str(Path(__file__).parent / "static"),           # Project images
5     str(Path(__file__).parent.parent / "shared-blocks" / "static"), # Shared images
6 ])
7
8
9 # Directory structure:
10 # documentation/manuals/
11 #   stx_manual_advanced/
12 #     static/
13 #       images/
14 #         advanced_banner.png    <- Local only
15 #         book.py
16 #   shared-blocks/
17 #     static/
18 #       images/
19 #         training_logo.png      <- Shared across projects
20 #   stx_manual_intro/
21 #     static/
22 #       images/
23 #         intro_banner.png       <- Local only
24 #         book.py

```

Pattern 2: Override + Base + Defaults

```

1 # Three-tier pattern: local overrides > team base > org defaults
2 stx.set_static_sources([
3     str(Path(__file__).parent / "static"),           # 1. Project overrides
4     str(Path(__file__).parent / "../..team/static"), # 2. Team-level assets
5 ])

```

```

6         str(Path(__file__).parent / "../../org/static"), # 3. Organization defaults
7     ])
8
9     # If both team and org have "header.png":
10    # -> team version is used (higher priority)
11    # If only org has "footer.png":
12    # -> org version is used (only match)

```

Pattern 3: Blocks Alongside Static

Combining LazyBlockRegistry + set_static_sources: When shared blocks reference images, both the block source and the static source must be configured. LazyBlockRegistry handles block discovery, while set_static_sources handles image discovery.

```

1  import streamtex as stx
2  from pathlib import Path
3
4  # Block sources (for LazyBlockRegistry)
5  shared_blocks_path = str(Path(__file__).parent.parent / "shared-blocks" / "blocks")
6  shared_blocks = stx.LazyBlockRegistry([shared_blocks_path])
7
8  # Static sources (for image/asset resolution)
9  stx.set_static_sources([
10     str(Path(__file__).parent / "static"),
11     str(Path(__file__).parent.parent / "shared-blocks" / "static"),
12 ])
13
14 # Now shared blocks can reference images by name,
15 # and the static resolver will find them in shared-blocks/static/

```

Fallback Behavior When File Is Not Found

Purpose

resolve_static() Fallback

No error raised: `resolve_static()` does NOT raise an exception when a file is not found. It silently returns the original relative path. This means errors may surface later when you try to open or read the file.

```
1  import streamtex as stx
2
3  # File does not exist in any static source
4  result = stx.resolve_static("nonexistent.json")
5  # result = "nonexistent.json" (original path returned as-is)
6
7  # The error surfaces when you try to use the path:
8  with open(result) as f:          # FileNotFoundError here!
9      data = json.load(f)
```

st_image() Fallback

Streamlit static fallback: When `st_image()` cannot find a file via static resolution, it falls back to Streamlit's built-in static serving path: `'app/static/images/{filename}'`. This requires `enableStaticServing = true` in `.streamlit/config.toml`.

```
1  # If "missing.png" is not found via static resolution:
2  st_image(uri="missing.png")
```

```
3 # -> Falls back to src="app/static/images/missing.png"
4 # -> This uses Streamlit's built-in static file serving
5 # -> Requires .streamlit/config.toml:
6 #     [server]
7 #     enableStaticServing = true
```

Details:

Best Practices for Organizing Static Assets

Purpose

1. Always call `set_static_sources()` in `book.py`

Configure sources at the entry point, before any blocks are loaded.
This ensures all blocks see the same resolution configuration.

2. Place `local static/` first in the sources list

The first source has the highest priority. Local assets should override shared ones, so list your project's `static/` directory first.

3. Use `images/` subdirectory for image assets

`st_image()` automatically searches the `images/` subdirectory within each static source. Organize images under `static/images/` for automatic discovery.

4. Name files with block prefixes to avoid collisions

Follow the naming convention: [block_filename]_image_[index].[ext].
For example, bck_welcome_image_01.png avoids name clashes across blocks.

5. Use `resolve_static()` for non-image assets

For data files (JSON, CSV, text), use `stx.resolve_static()` explicitly.
`st_image()` only handles image URIs — other asset types need manual resolution.

6. Debug with `get_static_sources()`

When assets are not found, print `stx.get_static_sources()` to verify the configured paths. Check that directories exist and contain the expected files.

```
1  import streamtex as stx
2
3  # Debugging: print the current static source configuration
4  print("Static sources:", stx.get_static_sources())
5  # -> ['abs/path/to/static', '/abs/path/to/shared-blocks/static']
6
7  # Debugging: check if a specific file resolves correctly
8  resolved = stx.resolve_static("images/logo.png")
9  print(f"Resolved: {resolved}")
10 # -> If found: '/abs/path/to/static/images/logo.png'
11 # -> If not found: 'images/logo.png' (original path)
12
13 import os
14 print(f"Exists: {os.path.exists(resolved)}")
```

Summary: Static resolution is a simple but powerful mechanism.
Configure once in `book.py`, use simple filenames in blocks,
and let StreamTeX handle the path resolution.
Combined with LazyBlockRegistry, it enables seamless asset sharing
across projects without duplicating files.

Block Helpers: 3 Usage Patterns

StreamTeX provides a hybrid block helpers system with 3 usage patterns. Choose the one that fits your needs, from simple to advanced.

Pattern 1: Config Injection (Recommended)

Purpose

How it works:

1. Define **ProjectBlockHelperConfig** with your styles
2. Call **set_block_helper_config()** once at startup
3. All **show_code()**, **show_explanation()**, etc. use your styles automatically

Code example:

```
1 from streamtex import BlockHelperConfig, set_block_helper_config
2 from custom.styles import Styles as s
3
4 class ProjectBlockHelperConfig(BlockHelperConfig):
5     def get_code_style(self):
6         return s.project.containers.code_box
7
```

[wrap](#)

```
8     def get_explanation_style(self):
9         return s.project.containers.explanation_box
10
11 set_block_helper_config(ProjectBlockHelperConfig())
12
13 # Now all helpers use your styles!
14 from blocks.helpers import show_code
15 show_code("print('hello')") # Uses s.project.containers.code_box
```

Benefits:

- **DRY**: Define styles once, use everywhere
- **Simple**: No parameters needed
- **Global**: Consistent styling across all blocks
- **Flexible**: Can still override per-call with `style=` parameter

Pattern 2: Standalone Functions (Simple)

Purpose

How it works:

1. Import helpers from **streamtex** or local blocks
2. Call them directly with optional **style=** override
3. Styles from config are used automatically

Code example:

```
1 from blocks.helpers import show_code, show_explanation
2 from custom.styles import Styles as s
3
4 # Uses config-injected style
5 show_code("print('hello')")
6
7 # Override with custom style
8 show_code("print('world')", style=s.custom.special_box)
9
10 # Standalone explanation
11 show_explanation('''
12 This shows how to use the simple pattern.
13 Just call the function!
14 ''')
```

Benefits:

- **Minimalist:** No classes or complexity
- **Flexible:** Can override style per-call
- **Intuitive:** Just call the function
- Works with Config Injection

Pattern 3: OOP Inheritance (Advanced)

Purpose

How it works:

1. Subclass **BlockHelper** or **ProjectBlockHelper**
2. Override methods to add custom logic
3. Add new project-specific methods
4. Instantiate and use in your blocks

Code example:

```
1  from streamtex import BlockHelper
2  from custom.styles import Styles as s
3
4  class MyCustomHelper(BlockHelper):
5      # Override parent method
6      def show_code(self, code, language="python"):
7          print(f>About to show code in {language}") # Custom logic!
8          return super().show_code(code, language)
9
10     # Add new project-specific method
11     def show_comparison(self, before, after):
12         '''Compare two code snippets.'''
13         st_write(s.large, "Before:")
14         self.show_code(before)
15         st_write(s.large, "After:")
16         self.show_code(after)
17
18
19 # Usage
20 helper = MyCustomHelper()
21
```

```
helper.show_code("print('hello')")  
helper.show_comparison("old_code()", "new_code()")
```

Benefits:

- **Extensible:** Add custom logic before/after parent calls
- **Organized:** Group related helpers in a class
- **Type-safe:** IDEs can autocomplete methods
- **Chainable:** Use `super()` to leverage parent behavior

Pattern 4: Expert (All 3 Combined)

Purpose

How it works:

1. **Config Injection** provides global styles (Pattern 1)
2. **OOP class** adds custom methods (Pattern 3)
3. **Function calls** can still override with `style=` (Pattern 2)
4. Chain everything together

Code example:

```
1 from blocks.helpers import ExpertBlockHelper
2
3 # ExpertBlockHelper combines all patterns
4 expert = ExpertBlockHelper()
5
6 # Uses config styles (DI)
7 expert.show_code("print('hello')")
8
9 # Custom method (OOP)
10 expert.show_advanced_comparison("old", "new")
11
12 # Runtime override (Function)
13 from streamtex import show_code
14 show_code("print('world')", style=s.custom.style)
```

When to use:

- Complex projects with varied requirements
- Need both global config + custom logic
- Runtime overrides for special cases
- Advanced developers who understand all patterns

Pattern Comparison

Details:

- **Pattern 1 (Config Injection)** — Complexity: Easy. Setup: One config class + `set_block_helper_config()`. Use when: Most projects, simple to medium complexity
- **Pattern 2 (Standalone Functions)** — Complexity: Very Easy. Setup: Just import and call. Use when: Minimal setup, direct function usage
- **Pattern 3 (OOP Inheritance)** — Complexity: Advanced. Setup: Subclass `BlockHelper`, override methods. Use when: Need custom logic, organize helpers
- **Pattern 4 (Expert)** — Complexity: Very Advanced. Setup: Config + Class + Runtime overrides. Use when: Maximum flexibility needed

Best Practices

Purpose

1. Start with Pattern 1 (Config Injection)

- It covers 90% of use cases
- Simple to understand and maintain

2. Add project-specific helpers

- Create helpers unique to your project
- Keep them in `blocks/helpers.py`

3. Only use OOP when you need it

- Don't over-engineer simple cases
- Use when you have custom logic to add

4. Document your helpers

- Add docstrings explaining purpose
- Show usage examples in docstrings

5. Keep helper functions pure

- Same input → same output
- No side effects (unless intentional)

Live Demonstration

Each pattern demonstrated with actual code below:

Demo 1: Simple Pattern

```
1 print('hello')
```

Demo 2: Config Injection Pattern

```
1 print('hello')
```

Demo 3: OOP Pattern

```
1 print('hello')
```

Comparison

Before:

```
1 old_code( )
```

After:

```
1 new_code( )
```

Demo 4: Expert Pattern (All Combined)

```
1 print('hello')
```

Summary

Purpose

Recommended path:

1. Start with **Config Injection** (Pattern 1)
2. Add project-specific helpers
3. Use **OOP** only if needed (Pattern 3)
4. Expert pattern for edge cases

Learn more:

- See `stx_manual_intro/blocks/helpers.py` for minimal example
- See `stx_manual_advanced/blocks/helpers.py` for complete showcase
- See `documentation/template_project/blocks/helpers.py` for annotated template

Next steps:

1. Try each pattern in a small block
2. Find your preferred pattern
3. Build your project's helpers

The Atomic Block Pattern

Purpose

Composite Blocks

Purpose

Atomic Blocks

Purpose

Directory Structure

```
blocks/  
├── bck_my_section.py          # Composite: loads and includes atoms  
└── _atomic/  
    ├── bck_topic_a.py        # Atomic: one topic  
    └── bck_topic_b.py        # Atomic: another topic
```

Composite Block Code

Purpose

```
1 import streamtex as stx
2 from streamtex import st_include
3
4 bck_topic_a = stx.load_atomic_block("bck_topic_a", __file__)
5 bck_topic_b = stx.load_atomic_block("bck_topic_b", __file__)
6
7 class BlockStyles:
8     pass
9
10 def build():
11     st_include(bck_topic_a)
12     st_include(bck_topic_b)
```

Atomic Block Code

Purpose

```
1 """Atomic block – Topic A."""
2
3 from streamtex import *
4 from streamtex.enums import Tags as t
5 from custom.styles import Styles as s
6 from blocks.helpers import show_code, show_explanation
7
8 class BlockStyles:
9     heading = s.project.titles.section_title + s.center_txt
10     sub = s.project.titles.section_subtitle
11
12 bs = BlockStyles
```

```
13
14 def build():
15     st_write(bs.heading, "Topic A", tag=t.div, toc_lvl="1")
16     st_space("v", 1)
    show_explanation("Content for topic A.")
```

Benefits

Purpose

When to Use

Details:

Overlays

st_overlay with positioned layers

Purpose

```
1 with st_overlay(s.container.sizes.width_full) as o:
2     with st_block(s.container.bg_colors.dark_slate_blue_bg
3         + ns("height: 200px; width: 100%;")):
4         st_write(s.text.colors.white + s.large,
5             "Base content area (dark background)")
6     with o.layer(top=20, left=20):
7         with st_block(bs.overlay_bg):
8             st_write(bs.overlay_text, "Top-Left")
9     with o.layer(bottom=20, right=20):
10         with st_block(bs.overlay_bg):
11             st_write(bs.overlay_text, "Bottom-Right")
```

Top-Left

Base content area (dark background)

Bottom-Right

Layer positioning: top, left, right, bottom

Purpose

Details:

Center positioning: overlaying at center

Purpose

```
1 with st_overlay(s.container.sizes.width_full) as o:
2     with st_block(s.container.bg_colors.light_blue_bg
3         + ns("height: 200px; width: 100%;")):
4         st_write(s.large, "Background")
5
6     # Center overlay
7     with o.layer(top="50%", left="50%"):
8         with st_block(ns("transform: translate(-50%, -50%);")):
9             st_write(bs.overlay_text, "Centered")
```

Multiple layers: stacking overlays

Purpose

```
1 with st_overlay(s.container.sizes.width_full) as o:
2     # Base layer
3     with st_block(s.container.bg_colors.dark_blue_bg
```

```

4         + ns("height: 150px;")):
5         st_write(s.text.colors.white, "Base Content")
6
7     # Layer 1: Top-left
8     with o.layer(top=10, left=10):
9         st_write(bs.overlay_text, "Layer 1")
10
11    # Layer 2: Top-right
12    with o.layer(top=10, right=10):
13        st_write(bs.overlay_text, "Layer 2")
14
15    # Layer 3: Bottom-center
16    with o.layer(bottom=10, left="50%"):
17        st_write(bs.overlay_text, "Layer 3")

```

Use case: Badges and sale labels

Purpose

```

1  with st_overlay(s.container.sizes.width_full) as o:
2      # Product card
3      with st_block(s.container.bg_colors.light_gray_bg
4                    + ns("height: 200px; width: 100%;")):
5          st_write(s.large, "Product Image Area")
6
7      # Sale badge (top-right)
8      with o.layer(top=-10, right=-10):
9          with st_block(ns("background: #FF4444; border-radius: 50%; "
10                           "width: 60px; height: 60px; "
11                           "display: flex; align-items: center;")):

```

```
12 st_write(ns("color: white; text-align: center;"),  
13          "SALE\n50%")
```

Use case: Hover tooltips and annotations

Purpose

```
1 with st_overlay(s.container.sizes.width_full) as o:  
2     # Main content  
3     with st_block(s.container.bg_colors.light_blue_bg  
4                   + ns("height: 100px; padding: 20px;")):  
5         st_write(s.large, "Feature List")  
6  
7     # Annotation arrows/labels  
8     with o.layer(top=30, right=10):  
9         st_write(ns("color: #FF6600; font-size: 12px;"),  
10                "← New feature")
```

Advanced: z-index and layering order

Purpose

```
1 # Default: each layer on top of previous  
2 with st_overlay(s.container.sizes.width_full) as o:  
3     # Base (z-index: auto)  
4     with st_block(...): pass  
5
```

```

6      # Layer 1 (z-index: auto, but defined later)
7      with o.layer(top=20, left=20):
8          st_write(..., "Layer 1")
9
10     # To control explicitly:
11     with o.layer(top=30, left=30):
12         with st_block(ns("z-index: 10;")):
13             st_write(..., "Higher z-index")

```

Responsive overlays: percentage positioning

Purpose

```

1  with st_overlay(s.container.sizes.width_full) as o:
2      with st_block(ns("height: 200px; background: #f0f0f0;")):
3          st_write(s.large, "Container (responsive)")
4
5      # Top-right corner (responsive to container size)
6      with o.layer(top="10%", right="10%"):
7          st_write(bs.overlay_text, "Responsive")
8
9      # Center (responsive)
10     with o.layer(top="50%", left="50%"):
11         st_write(ns("color: red;"), "Centered")

```

Best practices and limitations

DO:

- Use overlays for static decorative elements
- Position labels, badges, and annotations
- Layer related content (image + caption)
- Use percentage positioning for responsiveness
- Keep overlay content minimal (text or icons)

DON'T:

- Hide important content under overlays
- Create overlays that block navigation
- Use overlays for layouts (use `st_grid` instead)
- Put interactive elements in overlays (not clickable)
- Overlap too many layers (becomes unreadable)

Limitations:

- Overlays are not interactive (no click events)
- Complex nested overlays can be hard to maintain
- Sizing depends on base container height
- Export to HTML may not preserve all positioning

Details:

Visibility

Three visibility modes

Purpose		
Mode	CSS	Behavior
hidden	display: none	Removed from layout (no space)
visible	visibility: visible	Normal rendering
invisible	visibility: hidden	Space preserved, content invisible

Live demonstration

Purpose		
---------	--	--

```
1 # Visible
2 st_write(s.bold + s.large + s.project.colors.success_green,
3         "s.visibility.visible:")
4 st_space("v", 1)
```

wrap

```
5 with st_span():
6     with st_block(bs.demo_box
7                   + s.container.bg_colors.light_blue_bg):
8         st_write("Box A")
9     with st_block(bs.demo_box
10                  + s.container.bg_colors.light_green_bg
11                  + s.visibility.visible):
12         st_write("Box B (visible)")
13     with st_block(bs.demo_box
14                  + s.container.bg_colors.light_coral_bg):
15         st_write("Box C")
16 st_space("v", 2)
17
18
19 # Invisible (space preserved)
20 st_write(s.bold + s.large + s.project.colors.highlight_amber,
21         "s.visibility.invisible:")
22 st_space("v", 1)
23 with st_span():
24     with st_block(bs.demo_box
25                  + s.container.bg_colors.light_blue_bg):
26         st_write("Box A")
27     with st_block(bs.demo_box
28                  + s.container.bg_colors.light_green_bg
29                  + s.visibility.invisible):
30         st_write("Box B (invisible)")
31     with st_block(bs.demo_box
32                  + s.container.bg_colors.light_coral_bg):
33         st_write("Box C")
34 st_space("v", 2)
35
36
37 # Hidden (removed from layout)
38 st_write(s.bold + s.large + s.project.colors.warning_red,
39         "s.visibility.hidden:")
40 st_space("v", 1)
```

```
41 with st_span():
42     with st_block(bs.demo_box
43                   + s.container.bg_colors.light_blue_bg):
44         st_write("Box A")
45     with st_block(bs.demo_box
46                   + s.container.bg_colors.light_green_bg
47                   + s.visibility.hidden):
48         st_write("Box B (hidden)")
49     with st_block(bs.demo_box
50                   + s.container.bg_colors.light_coral_bg):
51         st_write("Box C")
```

s.visibility.visible:

Box A Box B (visible) Box C

s.visibility.invisible:

Box A Box C

s.visibility.hidden:

Box A Box C

Details:

Themes

How themes work

Purpose

```
1 # Theme overrides for dark mode.
2 # Keys are style_ids, values are replacement CSS strings.
3 dark = {
4     "primary_blue": "color: #7AB8F5;",
5     "good_example_bg": "background-color: rgba(39, 174, 96, 0.25);",
6     "explanation_label": "color: #7AB8F5; font-weight: bold; font-size: 24pt;",
7     "details_label": "color: #F5C842; font-weight: bold; font-size: 24pt;",
8     "code_box_bg": "background-color: rgba(74, 144, 217, 0.12);",
9 }
```

Activating a theme in book.py

Purpose

```
1 import streamtex.styles as sts
2 from custom.themes import dark
3 sts.theme = dark
```

This project's theme overrides

Purpose		
style_id	Original	Dark Override
primary_blue	#4A90D9	#7AB8F5 (lighter)
section_title	Full style (blue 80pt bold)	Lighter blue for dark bg
good_example_bg	15% opacity green	25% opacity (brighter on dark)

Details:

StyleGrid: Per-cell styling in themes

Purpose

```
1 from streamtex import st_grid, StyleGrid, Style as ns
2
3 # Create a grid with per-cell styling
4 # Cell (1,1) gets border, cell (2,2) gets background
5 cell_styles = StyleGrid()
6
```

wrap

```

7 cell_styles[(1, 1)] = ns("border: 2px solid #7AB8F5;")
8 cell_styles[(2, 2)] = ns("background: rgba(122, 184, 245, 0.1);")
9
10 with st_grid(cols="1fr 1fr", cell_styles=cell_styles):
11     with g.cell(): st_write(s.large, "Top-left")
12     with g.cell(): st_write(s.large, "Top-right (with border)")
13     with g.cell(): st_write(s.large, "Bottom-left")
14     with g.cell(): st_write(s.large, "Bottom-right (with bg)")

```

Theme composition and merging

Purpose

```

1 # Define a base theme
2 base_theme = {
3     "primary_blue": "color: #4A90D9;",
4     "primary_text": "color: #333333;",
5     "accent_green": "color: #2ECC71;",
6 }
7
8 # Dark variant overrides specific colors
9 dark_theme = {
10     **base_theme, # Inherit all base colors
11     "primary_blue": "color: #7AB8F5;", # Lighter blue
12     "primary_text": "color: #E0E0E0;", # Lighter text
13 }
14
15 # Activate in book.py
16 import streamtex.styles as sts
17 sts.theme = dark_theme

```

Dynamic theme switching

Purpose

```
import streamlit as st
1 import streamtex.styles as sts
2 from custom.themes import light_theme, dark_theme
3
4 if "theme" not in st.session_state:
5     st.session_state.theme = "dark"
6
7 col1, col2 = st.columns(2)
8 with col1:
9     if st.button("Light Theme"):
10         st.session_state.theme = "light"
11 with col2:
12     if st.button("Dark Theme"):
13         st.session_state.theme = "dark"
14
15 # Apply selected theme
16 if st.session_state.theme == "light":
17     sts.theme = light_theme
18 else:
19     sts.theme = dark_theme
20
21 # Now render blocks - they use the active theme
22 st_book(blocks=[...])
23
```

Creating custom themes: best practices

DO:

- Use consistent color palettes (same hue, different lightness)
- Override only what changes between themes (not everything)
- Use RGBA colors for backgrounds to support transparency
- Document which style_ids are overridden in comments
- Test both light and dark modes

DON'T:

- Hardcode black (#000000) or white (#FFFFFF)
- Override every single style_id (most work in both modes)
- Mix color schemes (don't mix warm and cool accents)
- Use opaque colors that hide content (use transparency)
- Assume all users have same display brightness

Details:

Indexed responsive font scale

A single base value drives every palier

The indexed scale is RELATIVE: a single `base_pt_desktop` (default 18pt at palier 7) is multiplied by 29 adimensional ratios to produce every desktop palier. Tablet and mobile columns are then derived uniformly via `tablet_scale` (0.85) and `mobile_scale` (0.70). Changing the base re-scales everything proportionally — no per-palier hand-tuning needed. Curves differ only in their ratio silhouette; they all anchor at palier 7 = `base_pt_desktop`.

Three access modes — same value

Tailwind alias

The most readable mode — 13 stable names mapped to specific paliers. Names never change even when the underlying curve does.

Subscript

Dynamic indexing — pick the palier at runtime, ideal for loops over heading levels.

```
wrap
for level in range(4):
    # 12=32pt,
    10=24pt, 8=20pt,
1   6=16pt
2       idx = 12 - level *
3       2
4
5   st_write(s.scale[idx],
           f'H{level+1} heading')
```

Direct attribute

Autocomplete-friendly named attribute on `Text.sizes` — useful when an editor's static analyser is the primary reader.

```
wrap
from streamtex.styles
import Text

1
2   st_write(Text.sizes.id
3   x_7, 'BASE = 18pt')
4   st_write(s.text.sizes.
   idx_12, 'Display =
   32pt')
```

wrap

```
st_write(s.text_xs,  
'Caption')      # idx_5  
= 14pt  
st_write(s.text_base,  
'Body')         # idx_7  
= 18pt (BASE)  
st_write(s.text_3xl,  
'Heading')      # idx_11  
= 28pt  
st_write(s.text_7xl,  
'Hero title')   # idx_16  
= 60pt
```

BODY_CENTRIC

BELL

The four named curves

Purpose

Intended use

General documents — Word/Docs lineage.
Fine at small sizes, coarser in display range.
Uniform visual ratio (~1.125) between paliers. Predictable for design systems.
Dense in the body range (14-32pt). Best for prose-heavy manuals.
Sparse at extremes, dense in the middle.
One giant hero + small captions for slides.

Ratios at key paliers (5 / 7 / 8 / 12 / 16)

0.78 / 1.00 / 1.11 / 1.78 / 3.33

0.79 / 1.00 / 1.13 / 1.80 / 2.88

0.89 / 1.00 / 1.11 / 1.56 / 3.11

0.33 / 1.00 / 1.33 / 1.78 / 10.89

29-palier visual reference (live)

Every palier rendered with its actual token. Resize the browser past 1024px or 480px to see the values shrink.

Aa idx_0 = 8pt	Aa idx_1 = 9pt	Aa idx_2 = 10pt	Aa idx_3 = 11pt
Aa idx_4 = 12pt	Aa idx_5 = 14pt	Aa idx_6 = 16pt	Aa idx_7 = 18pt ← BASE = base_pt_desktop = 18pt by default
Aa idx_8 = 20pt	Aa idx_9 = 22pt	Aa idx_10 = 24pt	Aa idx_11 = 28pt
Aa idx_12 = 32pt	Aa idx_13 = 36pt	Aa idx_14 = 40pt	Aa idx_15 = 48pt

Aa

idx_16 = 60pt

Aa

idx_17 = 72pt

Aa

idx_18 = 96pt

Aa

idx_19 = 128pt

Aa

idx_20 = 156pt

Aa

idx_21 = 168pt

Aa

idx_22 = 180pt

Aa

idx_23 = 188pt

Aa

idx_24 = 192pt

Aa

idx_25 = 194pt

Aa

idx_26 = 195pt

Aa

idx_27 = 196pt

Aa

Per-document configuration

idx_28 = 200pt

Purpose

```
1  from streamtex import st_book, ScaleConfig, ScaleCurve
2
3  # 1) Bigger desktop type — every palier follows on every breakpoint.
4  #     Use when the project ships at a larger reading size (e.g. projection,
5  #     accessibility-first sites).
6  st_book([blocks...], scale=ScaleConfig(base_pt_desktop=24))
7
8  # 2) Shift the anchor — palier other than 7 maps to base_pt_desktop.
9  #     Use when the project's natural body text is at a different index
10 #     than the default (rare; advanced).
11 st_book([blocks...], scale=ScaleConfig(base_idx=8))
12
13 # 3) Custom breakpoint shrink — tighter or looser tablet/mobile.
14 #     Defaults: tablet_scale=0.85, mobile_scale=0.70.
15 st_book([blocks...], scale=ScaleConfig(
16     tablet_scale=0.9,      # tablet keeps 90% of desktop (less shrink)
17     mobile_scale=0.65,     # mobile drops to 65% (more shrink)
18 ))
19
20
```

```
21 # 4) Different silhouette — same base, different ratio profile.
22 st_book([blocks...], scale=ScaleConfig(curve=ScaleCurve.GEOMETRIC))
23
24 # Combine any subset of knobs.
25 st_book([blocks...], scale=ScaleConfig(
26     curve=ScaleCurve.GEOMETRIC,
27     base_pt_desktop=24,
28     tablet_scale=0.9,
29     mobile_scale=0.65,
    ))
```

Out-of-range tolerance

Subscribe clamps silently

`s.scale[-5]` returns `s.scale[0]`. `s.scale[100]` returns `s.scale[28]`. The library debug-logs the clamp but never raises — your block keeps rendering even when an index drifts out of range.

Live clamp demo

`s.scale[-5] → clamped to s.scale[0]`

s.scale[100]

→ clamped

to s.scale[28]

Responsive — resize the browser

Three breakpoints out of the box

Desktop $\geq 1024\text{px}$ uses `base_pt_desktop` $\times$ ratios. Tablet $480\text{-}1024\text{px}$ applies `tablet_scale` (default 0.85) uniformly to every desktop palier. Mobile $< 480\text{px}$ applies `mobile_scale` (default 0.70). Resize this window past 1024 and 480 to see every palier above shrink in lockstep.

Identity table — three names, one palier

Tailwind alias

s.text_xs
s.text_sm
s.text_base
s.text_lg
s.text_xl
s.text_2xl
s.text_3xl
s.text_4xl
s.text_5xl
s.text_6xl
s.text_7xl
s.text_8xl
s.text_9xl

Subscript

s.scale[5]
s.scale[6]
s.scale[7]
s.scale[8]
s.scale[9]
s.scale[10]
s.scale[11]
s.scale[12]
s.scale[13]
s.scale[15]
s.scale[16]
s.scale[17]
s.scale[19]

Direct attribute

s.text.sizes.idx_5
s.text.sizes.idx_6
s.text.sizes.idx_7
s.text.sizes.idx_8
s.text.sizes.idx_9
s.text.sizes.idx_10
s.text.sizes.idx_11
s.text.sizes.idx_12
s.text.sizes.idx_13
s.text.sizes.idx_15
s.text.sizes.idx_16
s.text.sizes.idx_17
s.text.sizes.idx_19

pt (desktop @ base=18)

14pt
16pt
18pt (BASE)
20pt
22pt
24pt
28pt
32pt
36pt
48pt
60pt
72pt
128pt

Curves head-to-head — same content, four scales

The same headline + body + caption rendered four times. Each section overrides `--stx-scale-*` on its own container, so paliers visibly differ across curves while every other style stays identical. All 4 curves share **base_pt_desktop = 18** — they differ only in their ratio profile.

WORD_PROCESSOR — the default, balanced for general documents. Silhouette: fine at small, coarser at display.

Headline (word_processor)

Body paragraph at palier 5. Notice how the gap between this body text and the headline above changes from curve to curve — the headline palier moves while the body palier stays at the same index.

Caption at palier 2 — desktop pt = 10 (palier 5 body = 14pt, palier 16 headline = 60pt).

GEOMETRIC — uniform ratio ~1.125 per palier, predictable modular scale for design systems.

Headline (geometric)

Body paragraph at palier 5. Notice how the gap between this body text and the headline above changes from curve to curve — the headline palier moves while the body palier stays at the same index.

Caption at palier 2 — desktop pt = 9 (palier 5 body = 14pt, palier 16 headline = 64pt).

BODY_CENTRIC — dense in 14-32pt body range, sparse extremes. Prose-friendly.

Headline (body_centric)

Body paragraph at palier 5. Notice how the gap between this body text and the headline above changes from curve to curve — the headline palier moves while the body palier stays at the same index.

Caption at palier 2 — desktop pt = 13 (palier 5 body = 16pt, palier 16 headline = 56pt).

BELL — bell-shaped: sparse extremes, dense middle, plateau at top. Best for slides.

Headline (bell)

Body paragraph at palier 5. Notice how the gap between this body text and the headline above changes from curve to curve — the headline palier moves while the body palier stays at the same index.

Caption at palier 2 — desktop pt = 10 (palier 5 body = 15pt, palier 16 headline = 52pt).

Picking a curve

Pick a curve for SILHOUETTE; pick a base for OVERALL SIZE. WORD_PROCESSOR is safe for any mixed document.

GEOMETRIC is the right pick when designers will hand-tune palette and want predictable ratios. BODY_CENTRIC suits dense prose (manuals, reports). BELL is for slide decks with one hero title and tight body — extremes collapse to a single sky-high palier so the entire deck reads at one giant size. To shift every palier proportionally, change base_pt_desktop on the same ScaleConfig.

Automatic Hover Effects

Feature: StreamTeX automatically adds hover preview cards for any links in your content when you enable hover effects.

How Hover Preview Works

When you hover over a link with `hover=True` enabled, a preview card appears showing:

- Page title from meta tags (if available)
- Brief description
- Favicon from the domain
- Preview thumbnail (if available)

When to Use Hover Preview

Hover preview is enabled by default in `st_write()` when you include links.

Perfect for: Documentation, tutorials, references, and educational content where users benefit from quick previews of linked resources.

Live Demo: Hover Over Links

Try hovering over these links to see the preview card appear:

Popular StreamTeX Resources:

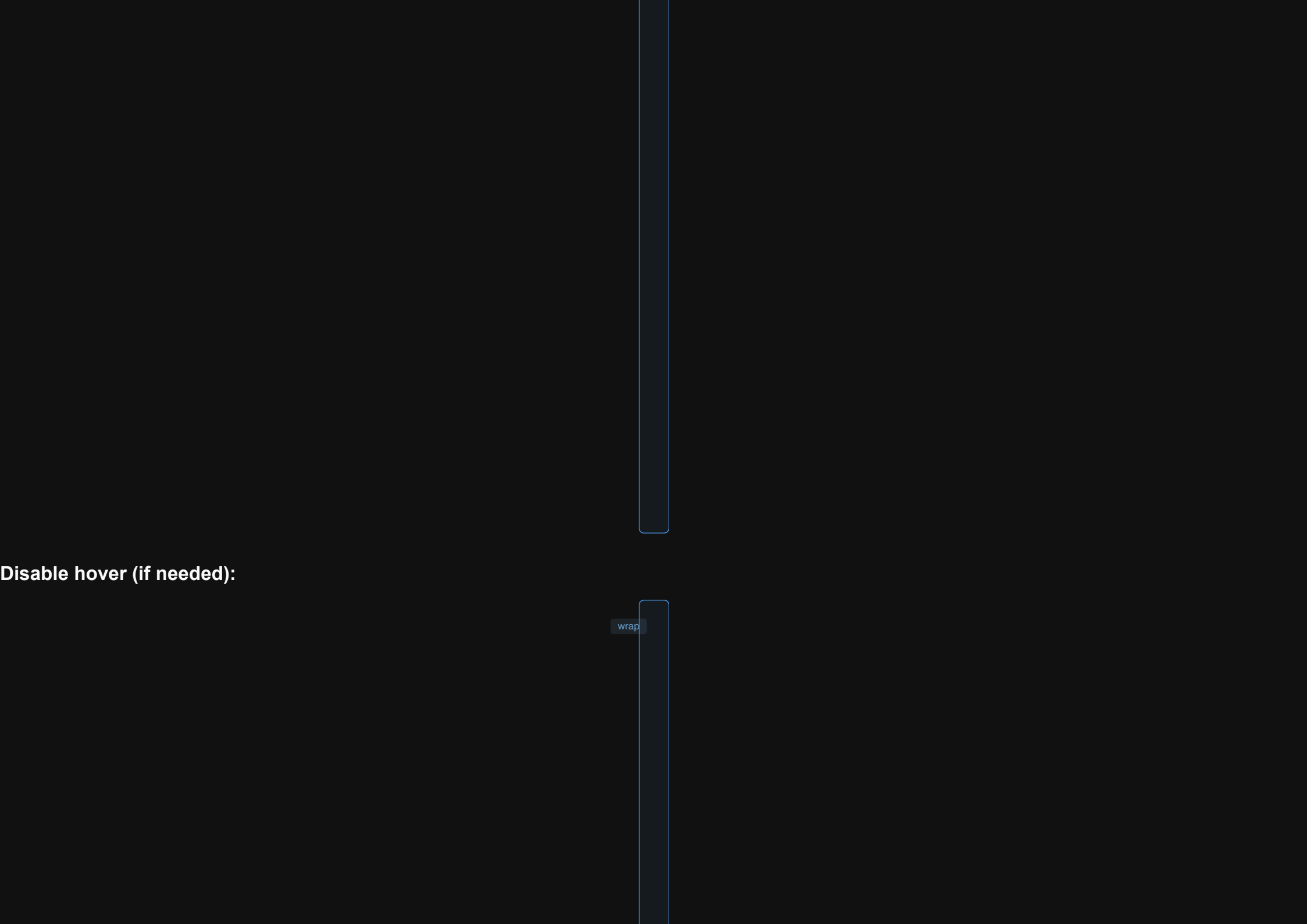
- [Documentation](#)
- [PyPI Package](#)

- [GitHub Repository](#)

Code Examples

Enable hover (default):

wrap



Disable hover (if needed):

wrap

Advanced: Links with Styling

Combine link styling with hover for rich interactive experiences:

wrap

LinkConfig API

Use LinkConfig to control link behavior globally:

wrap



LinkConfig controls the **target** attribute of all generated links. Internal links (same domain) and external links can have different targets.

Target options: `_self` (same tab), `_blank` (new tab), `_parent`, `_top`

Best Practices

1. Keep hover ON for external references and documentation links
2. Use underline or color to indicate links visually
3. Include meaningful link text (not 'click here')
4. Disable hover only for internal navigation or UI links
5. Use LinkConfig for global link target control

Tabs, Expanders & Popovers

Purpose

st.tabs

```
1 tab1, tab2, tab3 = st.tabs(["Text", "Grid", "List"])
2 with tab1:
3     with st_block(s.project.containers.tip_callout):
4         st_write(s.project.titles.tip_label, "Tip")
5         st_space("v", 1)
6         st_write(s.large, "Rich StreamTeX content inside a tab. "
7                 "All stx.* functions work here.")
8
9 with tab2:
10     with st_grid(cols=2, cell_styles=bs.cell) as g:
11         with g.cell():
12             st_write(s.large + s.bold, "Column A")
13         with g.cell():
14             st_write(s.large + s.bold, "Column B")
15         with g.cell():
16             st_write(s.large, "Data A1")
17         with g.cell():
18             st_write(s.large, "Data B1")
19
20 with tab3:
21     with st_list(list_type=lt.unordered, li_style=s.large) as l:
22         with l.item():
23             st_write("First item in a tab")
24         with l.item():
25             st_write("Second item in a tab")
```

```
with l.item():
    st_write("Third item in a tab")
```

Tip

Rich StreamTeX content inside a tab. All stx.* functions work here.

Column A	Column B
Data A1	Data B1

- First item in a tab
- Second item in a tab
- Third item in a tab

st.expander

```
1 with st.expander("Architecture Overview"):
2     with st_block(s.project.containers.good_callout):
3         st_write(s.project.titles.tip_label, "StreamTeX Architecture")
4         st_space("v", 1)
5         with st_list(list_type=lt.ordered, li_style=s.large) as l:
6             with l.item():
7                 st_write("book.py orchestrates the app")
8             with l.item():
9                 st_write("blocks/ contain the content modules")
10             with l.item():
```

wrap

```

12         st_write("custom/ holds project styles")
13
14     with st.expander("Style Examples"):
15         with st_block(s.project.containers.note_callout):
16             st_write(s.project.titles.details_label, "Composition")
17             st_space("v", 1)
18             st_write(s.large,
19                     "Styles compose with + operator: s.bold + s.Large")
20
21             st_br()
22             st_write(s.large,
23                     "Remove with - operator: style - s.bold")

```

StreamTeX Architecture

1. book.py orchestrates the app
2. blocks/ contain the content modules
3. custom/ holds project styles

Composition

Styles compose with + operator: s.bold + s.Large

Remove with - operator: style - s.bold

st.popover

```
1 with st.popover("Show info card"):  
2     with st_block(s.project.containers.result_box):  
3         st_write(s.large + s.bold + s.project.colors.primary_blue,  
4                 "StreamTeX Info")  
5         st_space("v", 1)  
6         st_write(s.large, "Version: 0.2.0")  
7         st_br()  
8         st_write(s.large, "Python >= 3.11")  
9         st_br()  
10        st_write(s.large, "Streamlit >= 1.54.0")
```

StreamTeX Info

- **Version: 0.2.0**
- **Python >= 3.11**
- **Streamlit >= 1.54.0**

Deep Nesting

Purpose

Nested A1

Nested B1

Cell A2

Cell B2

- Deeply nested item 1
- Deeply nested item 2

Details:

Dynamic & Conditional Content

Purpose

Style Switcher

```
1  STYLE_PRESETS = {
2      "Bold Blue": s.bold + s.Large + s.project.colors.primary_blue,
3      "Italic Teal": s.italic + s.Large + s.project.colors.accent_teal,
4      "Warning Red": s.bold + s.Large + s.project.colors.warning_red,
5      "Success Green": s.bold + s.Large + s.project.colors.success_green,
6      "Amber Highlight": s.bold + s.Large + s.project.colors.highlight_amber,
7  }
8
9
10 choice = st.selectbox("Choose a style preset",
11                       [*STYLE_PRESETS],
12                       key="bck30_style_select")
13 with st_block(s.project.containers.result_box):
14
```

```
st_write(STYLE_PRESETS[choice],  
         "Sample text with the selected style")
```

Sample text with the selected style

Conditional Rendering

```
1 coll1, col2 = st.columns(2)  
2 show_a = coll1.checkbox("Show block A", value=True,  
3                           key="bck30_show_a")  
4 show_b = col2.checkbox("Show block B", key="bck30_show_b")  
5  
6 if show_a:  
7     with st_block(s.project.containers.good_callout):  
8         st_write(s.project.titles.tip_label,  
9                 "Block A")  
10        st_space("v", 1)  
11        st_write(s.large,  
12                "Visible via Python conditional (if show_a).")  
13  
14 if show_b:  
15     with st_block(s.project.containers.tip_callout):  
16         st_write(s.project.titles.tip_label,  
17                 "Block B")  
18        st_space("v", 1)  
19        st_write(s.large,  
20                "Visible via Python conditional (if show_b).")  
21  
22 if not show_a and not show_b:  
23     with st_block(s.project.containers.note_callout):  
24         st_write(s.large, "No blocks selected. Check a box above.")
```

Block A

Visible via Python conditional (if show_a).

Dynamic Grid Builder

```
1 c1, c2 = st.columns(2)
2 grid_cols = c1.slider("Columns", 1, 5, 3, key="bck30_cols")
3 grid_rows = c2.slider("Rows", 1, 4, 2, key="bck30_rows")
4
5 with st_grid(cols=grid_cols, cell_styles=bs.cell) as g:
6     for i in range(grid_cols * grid_rows):
7         with g.cell():
8             st_write(s.large, f"Cell {i + 1}")
```

Cell 1

Cell 2

Cell 3

Cell 4

Cell 5

Cell 6

Progressive Disclosure (Wizard)

```
1 if "bck30_step" not in st.session_state:
2     st.session_state.bck30_step = 1
3 step = st.session_state.bck30_step
4
5 steps_data = [
6     ("Step 1: Choose your project name",
7
```

```

8         s.project.containers.tip_callout),
9         ("Step 2: Configure your styles",
10         s.project.containers.note_callout),
11         ("Step 3: Build your blocks",
12         s.project.containers.good_callout),
13     ]
14
15 for i, (label, callout) in enumerate(steps_data):
16     if step > i:
17         with st_block(callout):
18             st_write(s.large + s.bold, label)
19
20
21 col_prev, col_next, col_reset = st.columns(3)
22 if col_prev.button("Previous", key="bck30_prev",
23                   disabled=(step <= 1)):
24     st.session_state.bck30_step -= 1
25     st.rerun()
26 if col_next.button("Next", key="bck30_next",
27                   disabled=(step >= 3)):
28     st.session_state.bck30_step += 1
29     st.rerun()
30 if col_reset.button("Reset", key="bck30_reset"):
31     st.session_state.bck30_step = 1
32     st.rerun()

```

Step 1: Choose your project name

Details:

Marker Navigation

Purpose

Manual Markers — `st_marker()`

Use `st_marker()` to place a waypoint at any position in your content. By default markers are invisible (height 0, used only for navigation). Set `visible=True` to show a dashed border with the marker label.

```
1  from streamtex import st_marker
2
3  # Visible marker — dashed border + label text
4  st_marker("Section Start", visible=True)
5
6  # Invisible marker — zero height, navigation only
7  st_marker("Hidden Waypoint")
8
9  # Auto-generated label when omitted
10 st_marker() # label = "Marker N"
```

Live demo:

Visible Marker Demo

(An invisible marker was placed just above this line.)

Auto-Markers from TOC Headings

Instead of placing markers manually, you can bridge TOC headings to markers automatically via the `auto_marker_on_toc` setting in `MarkerConfig`:

```
1  from streamtex import MarkerConfig, st_book
2
3  # All TOC headings become markers
4  cfg = MarkerConfig(auto_marker_on_toc=True)
5
6  # Only levels 1 and 2 become markers
7  cfg = MarkerConfig(auto_marker_on_toc=2)
8
9  # Disabled (default) — no automatic bridge
10 cfg = MarkerConfig(auto_marker_on_toc=False)
11
12 st_book([...], marker_config=cfg)
```

Per-Heading Control — `marker=` Parameter

Even when `auto_marker_on_toc` is active, you can override the behavior for individual headings using the `marker=` parameter on `st_write`:

```
1 # Exclude a heading from markers (e.g. appendix)
2 st_write(s.huge, "Appendix", toc_lvl="1", marker=False)
3
4 # Force a heading to become a marker even when auto is off
5 st_write(s.huge, "Important Note", toc_lvl="2", marker=True)
```

Details:

MarkerConfig Reference

Purpose

Field Reference

show_nav_ui

Type: bool — **Default:** True

Show or hide the floating navigation widget (counter, prev/next buttons, popup list).

auto_marker_on_toc

Type: bool | int — **Default:** False

Bridge TOC headings to markers automatically. True = all levels, int N = up to level N, False = none. Recommended: 1 for paginated books.

nav_position

Type: str — **Default:** "bottom-right"

Widget position. Supported values: "bottom-right", "bottom-center".

nav_label_chars

Type: int — **Default:** 40

Max characters shown for the current marker label in the widget. Set to 0 to hide the label.

popup_open

Type: bool — **Default:** False

Initial state of the marker popup list (open or closed on first render).

next_keys

Type: list[str] — **Default:** ["PageDown"]

Keyboard keys to navigate to the next marker. Supports modifier syntax (see below).

prev_keys

Type: list[str] — **Default:** ["PageUp"]

Keyboard keys to navigate to the previous marker. Supports modifier syntax (see below).

draggable

Type: bool — **Default:** False

Allow the user to drag the floating navigation widget anywhere on screen. Position is persisted in localStorage.

collapsible

Type: bool — **Default:** False

Show a : button to collapse/expand the navigation widget. Collapsed state is persisted in localStorage.

Keyboard Modifier Syntax

Key definitions use the JavaScript **KeyboardEvent.key** values. Combine a modifier with a key using the + separator:

```
1  # Single keys
2  next_keys=["PageDown", "ArrowRight"]
3
4  # With modifiers: Ctrl, Shift, Alt, Meta (Cmd on macOS)
5  next_keys=["PageDown", "Ctrl+ArrowRight"]
```

```
6 prev_keys=["PageUp", "Ctrl+ArrowLeft"]
7
8 # Multiple modifiers are NOT chained — use one modifier per entry
```

Complete Example in book.py

```
1 from streamtex import st_book, MarkerConfig, BannerConfig
2
3 marker_config = MarkerConfig(
4     auto_marker_on_toc=1,          # Level-1 TOC headings = markers
5     nav_position="bottom-right",   # Widget in bottom-right corner
6     next_keys=["PageDown", "ArrowRight"],
7     prev_keys=["PageUp", "ArrowLeft"],
8     nav_label_chars=40,           # Show up to 40 chars of label
9     draggable=True,              # Drag the widget anywhere on screen
10    collapsible=True,             # : button to collapse/expand
11 )
12
13 st_book(
14     [...],
15     marker_config=marker_config,
16     paginate=True,
17     banner=BannerConfig.full(),   # Navigation banner style
18 )
```

Keyboard Safety

Details:

NumberingMode Enum Reference

Purpose

```
1  from streamtex import NumberingMode, TOCConfig
2
3  # SIDEBAR_ONLY — numbers appear only in the sidebar TOC (default)
4  toc = TOCConfig(numbering=NumberingMode.SIDEBAR_ONLY)
5
6  # BOTH — numbers appear both in sidebar and inline in the content
7  toc = TOCConfig(numbering=NumberingMode.BOTH)
8
9  # NONE — no numbering anywhere
10 toc = TOCConfig(numbering=NumberingMode.NONE)
```

Details:

Markers in Paginated Mode

Continuous vs Paginated

In continuous mode (`paginate=False`), navigating between markers triggers a smooth scroll to the target element on the same page.

In paginated mode (paginate=True), markers may span multiple pages. The navigation system handles page changes automatically:

Purpose

The Floating Widget

When show_nav_ui=True (default), a floating widget appears with the following elements:

- **Position counter** showing "current / total" (e.g. "3 / 12")
- **Prev** and **Next** buttons
- A **list button** that opens a popup with all markers
- An optional **label** showing the current marker name

The popup lists markers from all pages. In paginated mode, clicking a marker on a different page triggers a page change followed by a scroll to the target marker.

A scroll tracker updates the current marker as you scroll manually, keeping the counter and popup highlight in sync.

Architecture Overview

Details:

```
1  # Lifecycle in book.py (simplified)
2  reset_marker_registry(marker_config) # 1. Init registry
3
4  for block in blocks:                # 2. Render blocks
5      st_include(block)                # (st_write auto-registers markers)
6
7  inject_marker_navigation()           # 3. Emit widget + JS
```

[wrap](#)

Banner Configuration

Purpose

BannerMode

Purpose

BannerConfig Presets

Purpose

```
1  from streamtex import BannerConfig
2
3  # Full banner with all features enabled
4  banner = BannerConfig.full()
5
6  # Compact banner with slim layout
7  banner = BannerConfig.compact()
8
9  # Hidden banner (no banner rendered)
10 banner = BannerConfig.hidden()
```

[wrap](#)

Custom BannerConfig

Purpose

```
1  from streamtex import BannerConfig, BannerMode
2
3  banner = BannerConfig(
4      mode=BannerMode.FULL,
5      color="rgba(211, 47, 47, 0.8)",
6      show_title=True,
7      show_arrows=True,
8  )
```

What Each Mode Looks Like

Purpose

Usage in st_book()

Purpose

```
1  from streamtex import st_book, BannerConfig
2
3  st_book(
4      [bck_intro, bck_chapter_1, bck_chapter_2],
5      banner=BannerConfig.full(),
6  )
7
8
```

```
9  # Or with a custom config
10 st_book(
11     [bck_intro, bck_chapter_1, bck_chapter_2],
12     banner=BannerConfig(
13         mode=BannerMode.COMPACT,
14         color="rgba(25, 118, 210, 0.9)",
15         show_title=True,
16         show_dividers=False,
17     ),
18 )
```

Details:

st_chrome_banner()

Purpose

```
1  from streamtex import st_chrome_banner
2
3  # Called automatically by st_book() – no action needed.
4  # For standalone pages without st_book():
5  st_chrome_banner()
```

Details:

Inspector Configuration

Purpose

InspectorConfig Dataclass

Purpose

```
1  from streamtex import InspectorConfig
2
3  InspectorConfig(
4      enabled=True,           # Enable the inspector panel
5      password=None,         # Optional password protection
6      panel_width="35vw",    # CSS width of the sidebar panel
7      backup=True,           # Create .bak files before saving
8  )
```

Usage in st_book()

Purpose

```
1  from streamtex import st_book, InspectorConfig
2
3  st_book(
4      [bck_intro, bck_chapter_1, bck_chapter_2],
```

```
5     inspector=InspectorConfig(enabled=True),  
6 )
```

Inspector Features

Purpose

Panel Width Presets

Purpose

```
1 # Fixed-width panel  
2 InspectorConfig(enabled=True, panel_width="700px")  
3  
4 # Extra-large panel for wide monitors  
5 InspectorConfig(enabled=True, panel_width="1100px")
```

Password Protection

Purpose

```
1 # Password-protected inspector
2 InspectorConfig(
3     enabled=True,
4     password="secret",
5 )
```

Excluded Directories

Purpose

Details:

Purpose

What Are Collections?

A **Collection** is a hub for organizing and launching multiple StreamTeX projects.

It provides a centralized entry point where users can discover and access different courses, tutorials, or applications.

Each project is a complete, self-contained application (book, course, documentation) running on its own port.

Examples:

- Training platform with multiple courses
- Documentation portal with different docs
- Learning path with multiple modules

Collections Components

1. CollectionConfig - Configuration Class

```
1  from streamtex.collection import CollectionConfig, ProjectMeta
2
3  config = CollectionConfig(
4      title="My Learning Hub",
5      description="Curated courses for learning StreamTeX",
6      cards_per_row=3, # Number of cards per row in the grid
7  )
8
9  # Add projects to the collection
10 config.projects["intro-course"] = ProjectMeta(
11     title="Introduction to StreamTeX",
12     description="Learn the basics",
13     cover="static/images/covers/intro.png",
14     project_url="http://localhost:8502",
15     order=1,
16 )
```

2. ProjectMeta - Project Metadata

```
1 from streamtex.collection import ProjectMeta
2
3 project = ProjectMeta(
4     title="Advanced Python",          # Project name (displayed on card)
5     description="Deep dive into Python", # Brief overview
6     cover="static/images/cover.png",  # Path to thumbnail image
7     project_url="http://localhost:8503", # URL where project runs
8     order=1,                          # Sort order in the collection
9 )
```

3. st_collection() - Automatic UI

```
1 from streamtex import st_collection, CollectionConfig
2
3 config = CollectionConfig.from_toml("collection.toml")
4
5 # Automatic UI: renders title, description, and project cards grid
6 st_collection(config=config, home_styles=Styles)
```

Configuration Methods

Method 1: TOML Configuration File (Recommended)

Define your collection declaratively in a TOML file:

```
1 # collection.toml
2 [collection]
3 title = "StreamTeX Training Collection"
4 description = "Introduction, Advanced, and Deployment courses"
5 cards_per_row = 3
6
```

```

7  [projects.intro]
8  title = "Introduction to StreamTeX"
9  description = "Learn basics: text styling, containers, grids, layouts"
10 project_url = "http://localhost:8502"
11 order = 1
12
13 [projects.advanced]
14 title = "Advanced Features"
15 description = "Master advanced: shared blocks, multi-source, deployment"
16 project_url = "http://localhost:8503"
17 order = 2
18
19 [projects.deploy]
20 title = "Deployment Guide"
21 description = "Deploy: Docker, Streamlit Cloud, Hetzner/Coolify, GCP, CI/CD"
22 project_url = "http://localhost:8504"
23 order = 3

```

```

1  # book.py — load from TOML
2  from pathlib import Path
3  from streamtex.collection import CollectionConfig
4
5  config_path = Path(__file__).parent / "collection.toml"
6  config = CollectionConfig.from_toml(str(config_path))

```

Method 2: Programmatic (Python)

```

1  from streamtex.collection import CollectionConfig, ProjectMeta
2
3  config = CollectionConfig(
4      title="StreamTeX Learn",
5      description="Complete learning path",

```

```

6     cards_per_row=2,
7 )
8
9 config.projects["intro"] = ProjectMeta(
10     title="Introduction",
11     description="Get started with basics",
12     project_url="http://localhost:8502",
13     order=1,
14 )
15 config.projects["advanced"] = ProjectMeta(
16     title="Advanced",
17     description="Deep dive into concepts",
18     project_url="http://localhost:8503",
19     order=2,
20 )

```

Displaying Collections

Option 1: `st_collection()` — Automatic UI

Renders title, description, and a grid of project cards automatically:

```

1 from streamtex import st_collection
2 from custom.styles import Styles as s
3
4 st_collection(config=config, home_styles=s)

```

Option 2: `st_book()` — Custom Collection UI

For full control over layout and design, build your own collection blocks and wire them into `st_book()`:

```
# book.py – custom collection UI
1  from streamtex import st_book
2  import blocks
3
4  st_book([
5      blocks.bck_home,  # Your custom block with cards + styling
6  ], paginate=False)
7
8
9
10 # blocks/bck_home.py – custom card layout
11 import streamlit as st
12 from streamtex import st_grid, st_block, st_write, Style
13 from streamtex.collection import CollectionConfig
14
15 config = CollectionConfig.from_toml("collection.toml")
16
17 def build():
18     with st_grid(cols=config.cards_per_row, grid_style=Style("gap:24px;", "gap")):
19         for key, project in config.projects.items():
20             with st_block(card_style):
21                 st_write(s.Large, project.title)
22                 st_write(s.medium, project.description)
23                 st.link_button("Open", project.project_url)
```

Environment Variable URL Overrides

Project URLs from TOML can be overridden by environment variables. This is useful for deployment where URLs differ from localhost:

```
1  # Pattern: STX_URL_<PROJECT_KEY_UPPER>
2  # For [projects.test-intro], the env var is STX_URL_TEST_INTRO
3
```

```
4 # Example: set in Coolify env vars, Docker, or .env
5 STX_URL_TEST_INTRO=https://docs-intro.streamtex.org
6 STX_URL_TEST_ADVANCED=https://docs-advanced.streamtex.org
```

Project vs Collection

Single Project (st_book):

- One application/course/documentation
- Self-contained on one port
- Example: "Python Training Course"

Collection (st_collection or custom st_book):

- Multiple projects in one hub
- Project discovery and navigation
- Each project runs on its own port
- Example: "Training Platform" with Intro, Advanced, Deploy courses

When to use Collections:

- Multiple independent projects to manage
- Need centralized project hub

- Building platform (multi-course, multi-doc)

Best Practices

Configuration:

- Use TOML for static collections (easier to maintain)
- Use programmatic CollectionConfig for dynamic collections (loaded from database, API, etc.)
- Always specify **project_url** correctly (must match running port)

Content:

- Use meaningful project titles and descriptive descriptions
- Use cover images for visual appeal (recommended size: 300x200px)
- Set correct order (1, 2, 3...) to control project sequence

Deployment:

- Test all project URLs before deploying collection
- Use **STX_URL_*** environment variables for production URLs
- Each project must be running on its configured port

Details:

Markdown Rendering

Purpose

Basic Formatting

Purpose

```
1 stx.st_markdown(''  
2 # Welcome to StreamTeX  
3  
4 This is bold text and italic text. You can also use strikethrough.  
5  
6 - First item  
7 - Second item  
8   - Nested item  
9 - Third item  
10  
11 > This is a blockquote with rich formatting.  
12 ''')
```

Welcome to StreamTeX This is **bold text** and *italic text*. You can also use ~~strikethrough~~. - First item - Second item - Nested item - Third item > This is a blockquote with **rich** formatting.

Tables and Code Blocks

Purpose

```
1 stx.st_markdown(''  
2 ## Feature Comparison  
3  
4 | Feature | st_write() | st_markdown() |  
5 |-----|-----|-----|  
6 | Styled text | Native StreamTeX | Markdown syntax |  
7 | Tables | st_grid() | Pipe syntax |  
8 | Math | No |  $\dots$  and  $\dots$  |  
9 | File loading | No | file= parameter |  
10 ''')
```

```
## Feature Comparison | Feature | st_write() | st_markdown() | |-----|-----|-----| | Styled text | Native StreamTeX | Markdown syntax | | Tables | st_grid() | Pipe syntax | | Math | No |  $\dots$   
and  $\dots$  | | File loading | No | file= parameter |
```

LaTeX Math in Markdown

Purpose

```
1 stx.st_markdown(''  
2 ## Mathematics  
3  
4 Inline math: the formula  $E = mc^2$  changed physics forever.  
5  
6 Display math:  
7  
8 
$$\int_0^{\infty} e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$
  
9
```

```
10 The quadratic formula:  $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ 
11 '''
```

Mathematics Inline math: the formula $E = mc^2$ changed physics forever. Display math: $\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$ The quadratic formula: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

Styled Markdown

Purpose

```
1 # Wrap Markdown in a styled container
2 stx.st_markdown(
3     "# Styled Section\n\nThis content has **custom styling**.",
4     style=my_style,
5 )
```

Loading from Files

Purpose

```
1 # Load from a .md file in static/
2 stx.st_markdown(file="docs/readme.md")
3
4 # With styling and custom encoding
5 stx.st_markdown(file="docs/notes.md", style=my_style, encoding="utf-8")
```

Interactive Selection

```
1 examples = {"Basic": md1, "Tables": md2, "Math": md3, ...}
2 choice = st.selectbox("Choose an example", list(examples.keys()))
3 stx.st_markdown(examples[choice])
```

Welcome to StreamTeX This is **bold text** and *italic text*. You can also use ~~strikethrough~~. - First item - Second item - Nested item - Third item > This is a blockquote with **rich** formatting.

Details:

LaTeX Rendering

Purpose

When to Use What

Purpose

Simple: Math Formulas

Purpose

```

1 # Simple equation
2 stx.st_latex(r"E = mc^2")
3
4 # Integral
5 stx.st_latex(r"\int_0^\infty e^{-x^2}\,,dx = \frac{\sqrt{\pi}}{2}")
6
7 # Sum series
8 stx.st_latex(r"\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}")

```

$$E = mc^2$$

$$\int_0^\infty e^{-x^2}\,,dx = \frac{\sqrt{\pi}}{2}$$

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$$

Interactive Math Selection

$$E = mc^2$$

Document Fragments (st_latex_doc)

Purpose

```

1 stx.st_latex_doc(r""
2 \section{Introduction}
3 This is a simple LaTeX fragment rendered by \textbf{LaTeX.js}.
4 It supports \emph{emphasis}, \texttt{monospace}, and basic formatting.
5 """, height=150)

```

Coexistence with StreamTeX

Purpose

```
1 # Mix StreamTeX text, math, and document rendering
2 st_write(s.Large, "Introduction to Linear Algebra")
3 stx.st_latex(r"\mathbf{A}\vec{x} = \vec{b}")
4 stx.st_latex_doc(r'''
5     \section{Matrix Operations}
6     Solving $\mathbf{A}\vec{x} = \vec{b}$ requires...
7 ''', height=200)
```

Parsing Utilities

Purpose

```
1 from streamtex import extract_tikz, extract_math, extract_frames
2
3 # Extract all TikZ diagrams
4 tikz_blocks = extract_tikz(latex_source)
5
6 # Extract all math formulas ($, $$, \[, \()
7 math_exprs = extract_math(latex_source)
8
9
10
```

```
# Extract all Beamer frames  
frames = extract_frames(beamer_source)
```

Details:

Syntax Highlighting

Supported languages (Pygments)

Purpose

Top 20 of 595 languages

Purpose

Basic Usage

Purpose

```
1 st_latex_doc(r"""
2 \documentclass{article}
3 \begin{document}
4 \section{Introduction}
5 Hello from LaTeX!
6 \end{document}
7 """)
```

wrap

```
\documentclass{article} \begin{document} \section{Introduction} Hello from LaTeX! \end{document}
```

Fragment Wrapping

Purpose

```
1 # No \documentclass needed — StreamTeX wraps it automatically
2 st_latex_doc(r"""
3 \section{Quick Note}
4 This fragment is auto-wrapped in an article document.
```

wrap

```
5 Supports \textbf{bold}, \emph{italic}, and math:  $E = mc^2$ .
6 """, height=180)
```

```
\documentclass{article} \begin{document} \section{Quick Note} This fragment is auto-wrapped in an article document. Supports \textbf{bold}, \emph{italic}, and math:  $E = mc^2$ . \end{document}
```

Parameters

Purpose

- **style** — optional Style wrapper for the container
- **light_bg** — True for white background (default), False for dark/transparent
- **height** — iframe height in pixels (default 600)
- **hyphenate** — True for LaTeX.js hyphenation (default True)
- **file** — path to a .tex file (resolved via static assets)
- **encoding** — file encoding (default utf-8)

```
1 # Customizing rendering parameters
2 st_latex_doc(
3     latex_source,
4     height=400,           # taller iframe
5     light_bg=False,       # transparent background
6     hyphenate=False,      # disable hyphenation
7 )
```

Loading from File

Purpose

```
1 # Load a LaTeX document from a file
2 st_latex_doc(file="documents/my_article.tex")
3
4 # With custom encoding
5 st_latex_doc(file="documents/french_text.tex", encoding="latin-1")
```

LaTeX.js Capabilities

Purpose

- Sectioning: `\section`, `\subsection`, `\subsubsection`
- Text formatting: `\textbf`, `\emph`, `\texttt`, `\underline`
- Lists: `\begin{itemize}`, `\begin{enumerate}`
- Math: inline $\$...\$$ and display $\$...\$$ environments
- Quotes: `\begin{quote}`
- Footnotes: `\footnote{...}`

```
1 st_latex_doc(r"""
2 \section{Structured Content}
3 \begin{itemize}
4   \item First item with \textbf{bold}
5   \item Second with inline math:  $a^2 + b^2 = c^2$ 
6 \end{itemize}
7
8 \begin{quote}
9   A famous result:  $E = mc^2$ 
10 \end{quote}
11
12 \begin{enumerate}
13   \item Alice scored \textbf{95}
```

```
15     \item Bob scored \textbf{87}
16 \end{enumerate}
    """, height=350)
```

```
\documentclass{article} \begin{document} \section{Structured Content} \begin{itemize} \item First item with \textbf{bold} \item Second with inline math:  $a^2 + b^2 = c^2$  \end{itemize} \begin{quote} A famous result:  $E = mc^2$  \end{quote} \begin{enumerate} \item Alice scored \textbf{95} \item Bob scored \textbf{87} \end{enumerate} \end{document}
```

LaTeX.js Limitations

Purpose

- No custom packages (`\usepackage` is limited to built-in ones)
- No TikZ diagrams — use `st_tikz()` instead
- No BibTeX / bibliography management
- No `\def` or `\newcommand` macros
- No cross-references (`\ref`, `\label`)

Choosing the Right Function

Purpose

- `st_latex()` — math formulas only (KaTeX, fast, Streamlit native)
- `st_latex_doc()` — full documents, fragments, structured content (LaTeX.js, `iframe`)
- `st_tikz()` — TikZ diagrams (requires local `pdflatex` + `dvisvgm`)

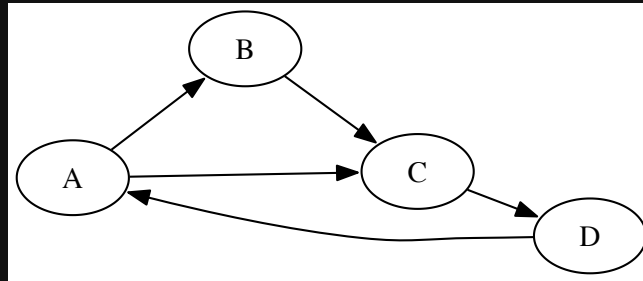
Details:

Graphviz Diagrams

Purpose

Simple Directed Graph

```
1 stx.st_graphviz(file="diagrams/simple_graph.dot")
```

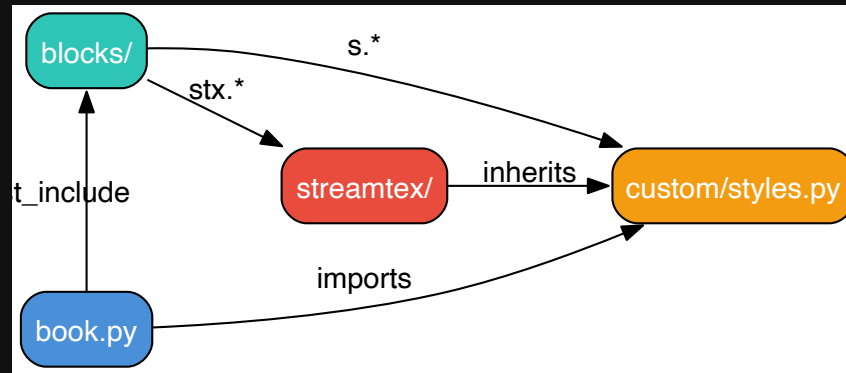


Source: diagrams/simple_graph.dot

```
1 digraph {
2     rankdir=LR
3     ranksep=0.4
4     nodesep=0.4
5     A -> B
6     B -> C
7     C -> D
8     D -> A
9     A -> C
10 }
```

Styled Flowchart

```
1 stx.st_graphviz(file="diagrams/flowchart.dot", height=600)
```



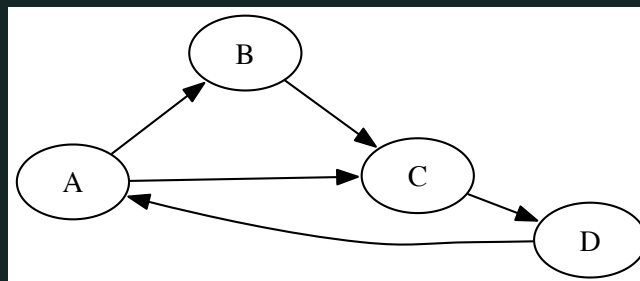
Source: diagrams/flowchart.dot

```
1 digraph {
2     rankdir=LR
3     ranksep=0.5
4     nodesep=0.5
5     node [shape=box, style="rounded,filled", fontname="Helvetica"]
6     edge [fontname="Helvetica"]
7
8     blocks [label="blocks/", fillcolor="#2EC4B6", fontcolor="white"]
9     book [label="book.py", fillcolor="#4A90D9", fontcolor="white"]
10    styles [label="custom/styles.py", fillcolor="#F39C12", fontcolor="white"]
11    streamtex [label="streamtex/", fillcolor="#E74C3C", fontcolor="white"]
12
13    {rank=same; blocks; book}
14
15    book --> blocks [label="st_include", constraint=false]
16    book --> styles [label="imports"]
17    blocks --> streamtex [label="stx.*"]
18    blocks --> styles [label="s.*"]
19 }
```

```
19 streamtex -> styles [label="inherits"]
20 }
```

Interactive Diagram Selection

```
1 DIAGRAMS = {
2     "Simple Graph": "diagrams/simple_graph.dot",
3     "StreamTeX Flowchart": "diagrams/flowchart.dot",
4     "Architecture": "diagrams/architecture.dot",
5     "Data Flow": "diagrams/data_flow.dot",
6 }
7
8 choice = st.selectbox("Choose a diagram", [*DIAGRAMS])
9 stx.st_graphviz(file=DIAGRAMS[choice])
```



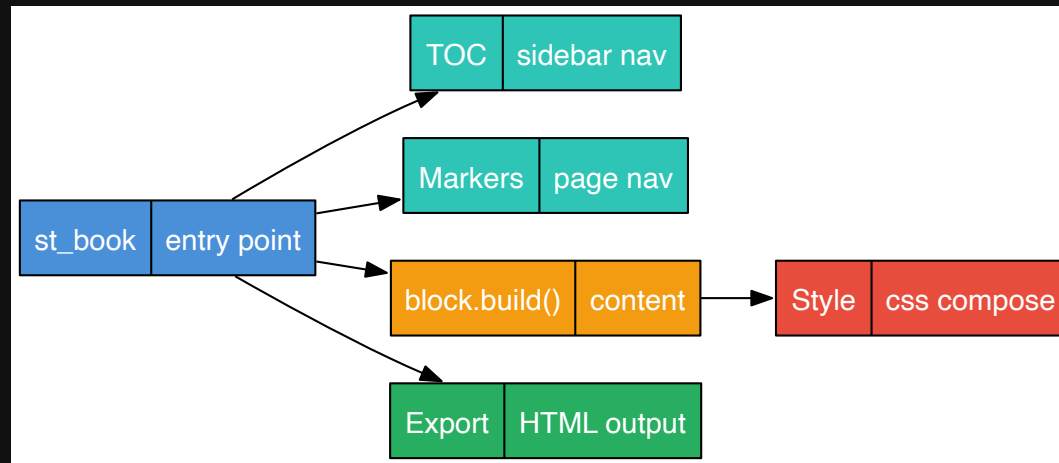
Source: diagrams/simple_graph.dot

```
1 digraph {
2     rankdir=LR
3     ranksep=0.4
4     nodesep=0.4
5     A -> B
6     B -> C
7     C -> D
```

```
8      D -> A
9      A -> C
10 }
```

Architecture Diagram

Purpose



Source: diagrams/architecture.dot

```
1 digraph {
2     rankdir=LR
3     ranksep=0.5
4     nodesep=0.3
5     node [shape=record, style=filled, fontname="Helvetica"]
6
7     st_book [label="{st_book|entry point}", fillcolor="#4A90D9", fontcolor="white"]
8     toc [label="{TOC|sidebar nav}", fillcolor="#2EC4B6", fontcolor="white"]
9     marker [label="{Markers|page nav}", fillcolor="#2EC4B6", fontcolor="white"]
10    block [label="{block.build()|content}", fillcolor="#F39C12", fontcolor="white"]
11 }
```

```
12 style [label="{Style|css compose}", fillcolor="#E74C3C", fontcolor="white"]
13 export [label="{Export|HTML output}", fillcolor="#27AE60", fontcolor="white"]
14
15 st_book -> toc
16 st_book -> marker
17 st_book -> block
18 block -> style
19 st_book -> export
}
```

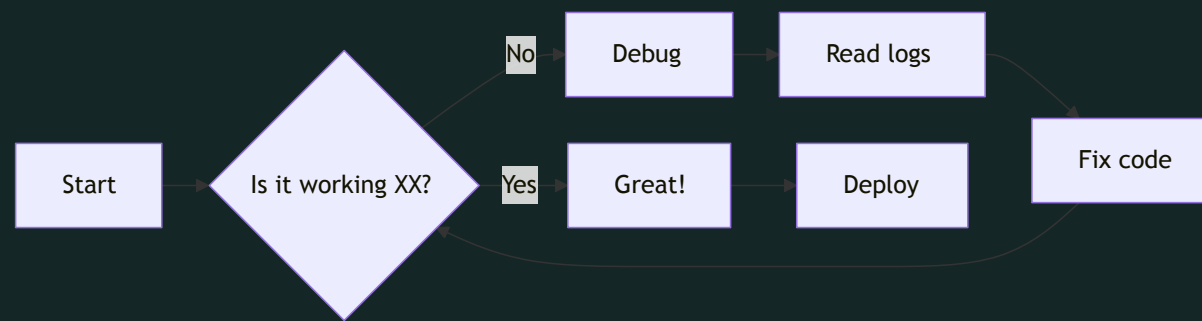
Details:

Mermaid Diagrams

Purpose

Flowchart

```
1 stx.st_mermaid(file="diagrams/flowchart.mmd", height=1000)
```



Source: *diagrams/flowchart.mmd*

```
1 %%{init: {'flowchart': {'rankSpacing': 30, 'nodeSpacing': 30}}}%  
2 graph LR  
3   A[Start] --> B{Is it working XX?}  
4   B -->|Yes| C[Great!]  
5   B -->|No| D[Debug]  
6   D --> E[Read logs]  
7   E --> F[Fix code]  
8   F --> B  
9   C --> G[Deploy]
```

Sequence Diagram

Purpose

```
1 stx.st_mermaid(file="diagrams/sequence.mmd", height=1000)
```



Source: *diagrams/sequence.mmd*

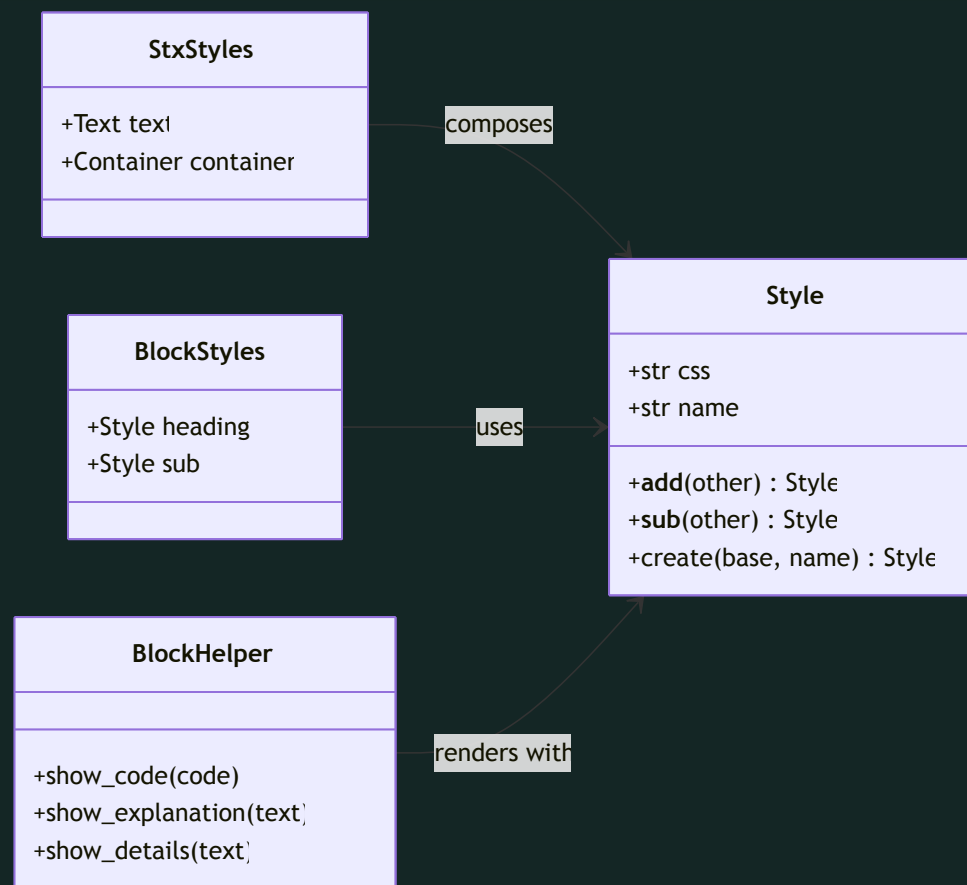
```
1 sequenceDiagram
2   participant U as User
3   participant B as Browser
4   participant S as Streamlit
5   participant SX as StreamTeX
6
```

wrap

```
7      U->>B: Open app
8      B->>S: HTTP request
9      S->>SX: st_book(blocks)
10     SX->>SX: Load TOC + Markers
11     SX->>S: Render HTML
12     S->>B: Response
13     B->>U: Display page
14     U->>B: Navigate (PageDown)
15     B->>S: Rerun
16     S->>SX: Render next block
17     SX->>S: HTML fragment
18     S->>B: Updated page
```

Class Diagram

Purpose



Source: *diagrams/class_diagram.mmd*

```

1  %%{init: {'class': {'rankSpacing': 30, 'nodeSpacing': 20}}}%
2  classDiagram
3      direction LR
4      class Style {
5          +str css
6          +str name
7          +__add__(other) Style
8          +__sub__(other) Style
9          +create(base, name) Style
10     }
11     class StxStyles {
12

```

wrap

```

13         +Text text
14         +Container container
15     }
16     class BlockStyles {
17         +Style heading
18         +Style sub
19     }
20     class BlockHelper {
21         +show_code(code)
22         +show_explanation(text)
23         +show_details(text)
24     }
25
26
27     StxStyles --> Style : composes
28     BlockStyles --> Style : uses
29     BlockHelper --> Style : renders with

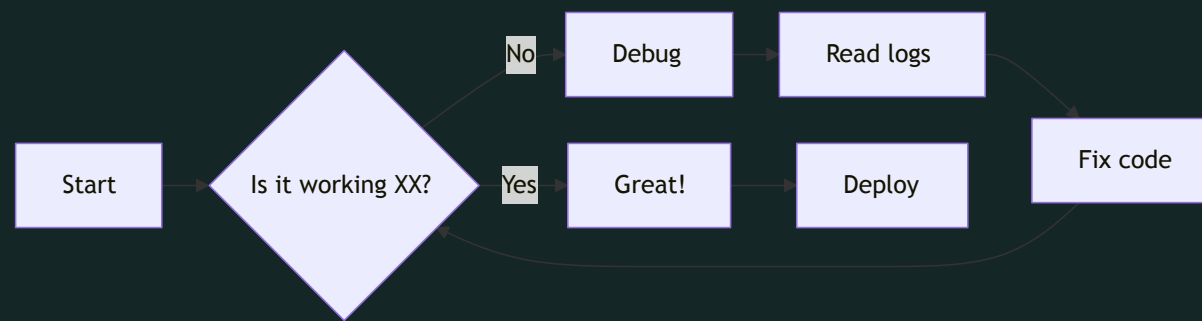
```

Interactive Diagram Selection

```

1  DIAGRAMS = {
2      "Flowchart": "diagrams/flowchart.mmd",
3      "Sequence Diagram": "diagrams/sequence.mmd",
4      "Class Diagram": "diagrams/class_diagram.mmd",
5  }
6
7  choice = st.selectbox("Choose a Mermaid diagram", [*DIAGRAMS])
8  stx.st_mermaid(file=DIAGRAMS[choice], height=1000)

```



Source: *diagrams/flowchart.mmd*

```
1  %%{init: {'flowchart': {'rankSpacing': 30, 'nodeSpacing': 30}}}%  
2  graph LR  
3      A[Start] --> B{Is it working XX?}  
4      B -->|Yes| C[Great!]  
5      B -->|No| D[Debug]  
6      D --> E[Read logs]  
7      E --> F[Fix code]  
8      F --> B  
9      C --> G[Deploy]
```

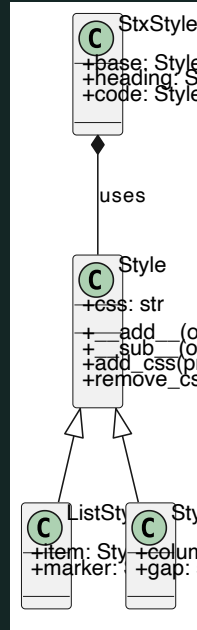
Details:

PlantUML Diagrams

Purpose

Class Diagram

```
1 stx.st_plantuml(file="diagrams/class_diagram.puml", height=1000)
```



Source: diagrams/class_diagram.puml

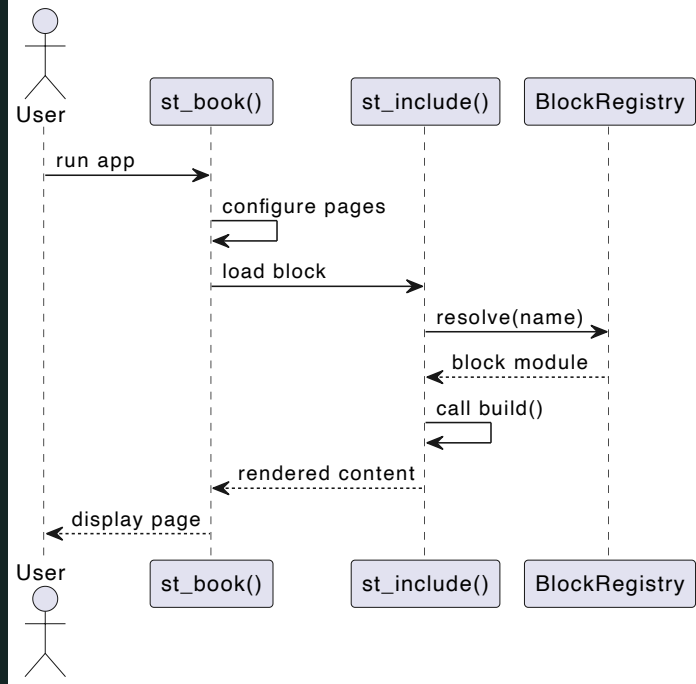
```
1 @startuml
2 skinparam classAttributeIconSize 0
3 skinparam shadowing false
4
5 class Style {
6     +css: str
7     +__add__(other): Style
8     +__sub__(other): Style
9     +add_css(prop, val): Style
10    +remove_css(prop): Style
11 }
12
13
14 class ListStyle {
15     +item: Style
```

```
16     +marker: Style
17 }
18
19 class StyleGrid {
20     +columns: list[Style]
21     +gap: str
22 }
23
24 class StxStyles {
25     +base: Style
26     +heading: Style
27     +code: Style
28 }
29
30
31 Style <|-- ListStyle
32 Style <|-- StyleGrid
33 StxStyles *-- Style : uses
34 @enduml
```

Sequence Diagram

Purpose

```
1 stx.st_plantuml(file="diagrams/sequence_diagram.puml", height=1000)
```



Source: *diagrams/sequence_diagram.puml*

```

1  @startuml
2  skinparam shadowing false
3
4  actor User
5  participant "st_book()" as Book
6  participant "st_include()" as Include
7  participant "BlockRegistry" as Registry
8
9
10 User -> Book: run app
11 Book -> Book: configure pages
12 Book -> Include: load block
13 Include -> Registry: resolve(name)
14 Registry --> Include: block module
15 Include -> Include: call build()
16 Include --> Book: rendered content
17

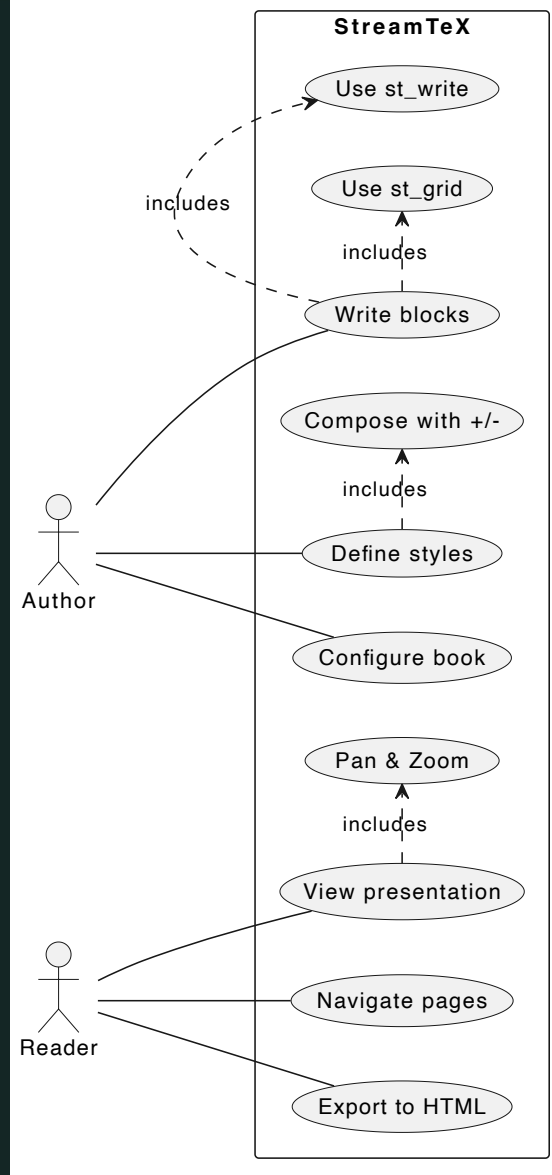
```

wrap

```
Book --> User: display page
@enduml
```

Use Case Diagram

Purpose



Source: diagrams/usecase_diagram.puml

```
1 @startuml
2 skinparam shadowing false
3
4 left to right direction
5
6
```

wrap

```

7 actor Author
8 actor Reader
9
10 rectangle StreamTeX {
11     Author -- (Write blocks)
12     Author -- (Define styles)
13     Author -- (Configure book)
14     (Write blocks) .> (Use st_write) : includes
15     (Write blocks) .> (Use st_grid) : includes
16     (Define styles) .> (Compose with +/-) : includes
17     Reader -- (View presentation)
18     Reader -- (Navigate pages)
19     Reader -- (Export to HTML)
20     (View presentation) .> (Pan & Zoom) : includes
21 }
@enduml

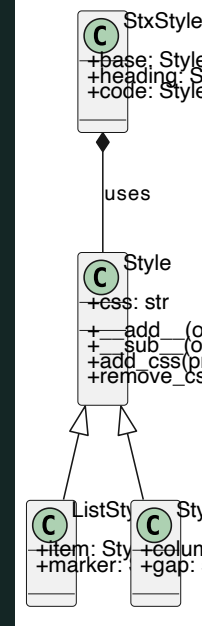
```

Interactive Diagram Selection

```

1 DIAGRAMS = {
2     "Class Diagram": "diagrams/class_diagram.puml",
3     "Sequence Diagram": "diagrams/sequence_diagram.puml",
4     "Use Case Diagram": "diagrams/usecase_diagram.puml",
5 }
6
7 choice = st.selectbox("Choose a PlantUML diagram", [*DIAGRAMS])
8 stx.st_plantuml(file=DIAGRAMS[choice], height=1000)

```



Source: *diagrams/class_diagram.puml*

```

1  @startuml
2  skinparam classAttributeIconSize 0
3  skinparam shadowing false
4
5  class Style {
6      +css: str
7      +__add__(other): Style
8      +__sub__(other): Style
9      +add_css(prop, val): Style
10     +remove_css(prop): Style
11 }
12
13
14 class ListStyle {
15     +item: Style
16     +marker: Style
17 }
18
19

```

wrap

```
20 class StyleGrid {
21   +columns: list[Style]
22   +gap: str
23 }
24
25 class StxStyles {
26   +base: Style
27   +heading: Style
28   +code: Style
29 }
30
31 Style <|-- ListStyle
32 Style <|-- StyleGrid
33 StxStyles *-- Style : uses
34 @enduml
```

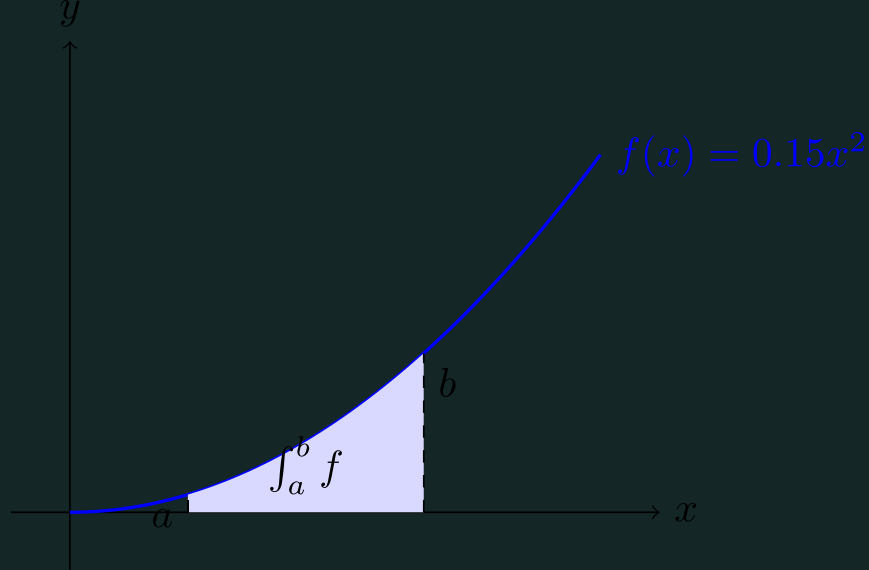
Details:

TikZ Diagrams

Purpose

Function Plot

```
1 stx.st_tikz(file="diagrams/simple_shapes.tex", height=800)
```



Source: diagrams/simple_shapes.tex

```

1 \begin{tikzpicture}
2   % Axes
3   \draw[>-] (-0.5,0) -- (5,0) node[right] {$x$};
4   \draw[>-] (0,-0.5) -- (0,4) node[above] {$y$};
5
6   % Curve
7   \draw[blue, thick, domain=0:4.5, samples=100]
8     plot (\x, {0.15*\x*\x}) node[right] {$f(x) = 0.15x^2$};
9
10  % Filled area
11  \fill[blue!15, domain=1:3, samples=50]
12    (1,0) -- plot (\x, {0.15*\x*\x}) -- (3,0) -- cycle;
13
14  % Bounds
15  \draw[dashed] (1,0) -- (1,0.15) node[below left] {$a$};
16  \draw[dashed] (3,0) -- (3,1.35) node[below right] {$b$};
17
18  % Label
19
20

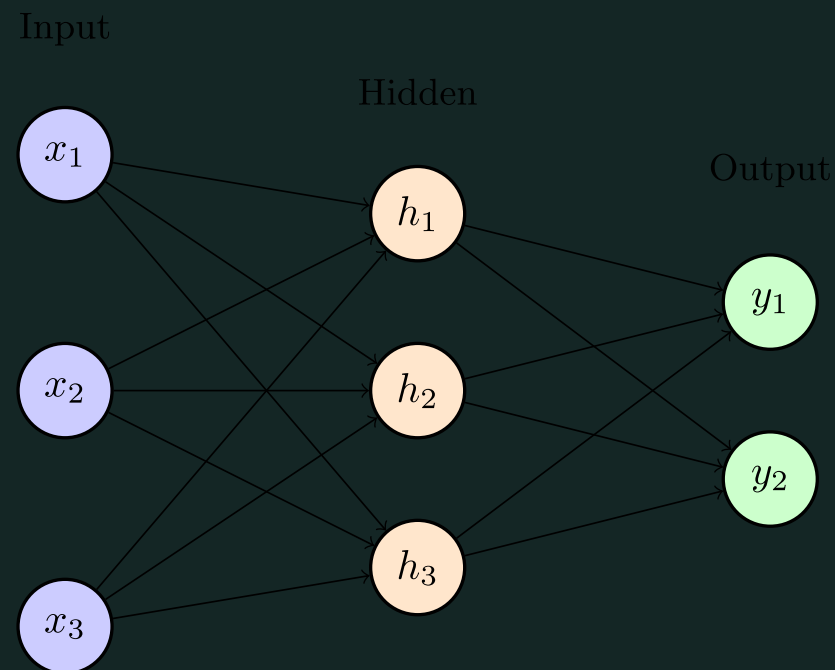
```

```
\node at (2,0.4) {\int_a^b f$};  
\end{tikzpicture}
```

Neural Network

Purpose

```
1 stx.st_tikz(file="diagrams/neural_network.tex", height=800)
```



Source: diagrams/neural_network.tex

```
1 \begin{tikzpicture}[  
2   node distance=1.5cm,  
3   neuron/.style={circle, draw, minimum size=0.8cm, thick},
```

```

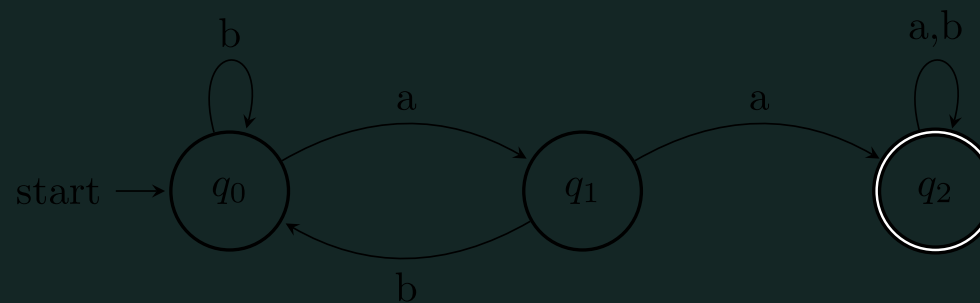
4   input/.style={neuron, fill=blue!20},
5   hidden/.style={neuron, fill=orange!20},
6   output/.style={neuron, fill=green!20},
7 ]
8   % Input layer
9   \node[input] (i1) at (0,2) {$x_1$};
10  \node[input] (i2) at (0,0) {$x_2$};
11  \node[input] (i3) at (0,-2) {$x_3$};
12
13  % Hidden layer
14  \node[hidden] (h1) at (3,1.5) {$h_1$};
15  \node[hidden] (h2) at (3,0) {$h_2$};
16  \node[hidden] (h3) at (3,-1.5) {$h_3$};
17
18  % Output layer
19  \node[output] (o1) at (6,0.75) {$y_1$};
20  \node[output] (o2) at (6,-0.75) {$y_2$};
21
22
23  % Connections input -> hidden
24  \foreach \i in {1,2,3}
25      \foreach \h in {1,2,3}
26          \draw[->] (i\i) -- (h\h);
27
28  % Connections hidden -> output
29  \foreach \h in {1,2,3}
30      \foreach \o in {1,2}
31          \draw[->] (h\h) -- (o\o);
32
33
34  % Labels
35  \node[above=0.3cm] at (0,2.5) {\small Input};
36  \node[above=0.3cm] at (3,2) {\small Hidden};
   \node[above=0.3cm] at (6,1.3) {\small Output};
\end{tikzpicture}

```

Finite Automaton (with preamble)

Purpose

```
1 stx.st_tikz(  
2   file="diagrams/finite_automaton.tex",  
3   height=800,  
4   preamble=r"\usetikzlibrary{automata,positioning}",  
5 )
```



Source: diagrams/finite_automaton.tex

```
1 \begin{tikzpicture}[  
2   ->, >=stealth, shorten >=1pt,  
3   node distance=3cm,  
4   state/.style={circle, draw, minimum size=1cm, thick},  
5   accepting/.style={state, double},  
6 ]  
7   \node[state, initial] (q0) {$q_0$};  
8   \node[state] (q1) [right of=q0] {$q_1$};  
9   \node[accepting] (q2) [right of=q1] {$q_2$};  
10  
11
```

```

12 \path (q0) edge [bend left] node [above] {a} (q1)
13      (q0) edge [loop above] node {b} ()
14      (q1) edge [bend left] node [above] {a} (q2)
15      (q1) edge [bend left] node [below] {b} (q0)
16      (q2) edge [loop above] node {a,b} ();
\end{tikzpicture}

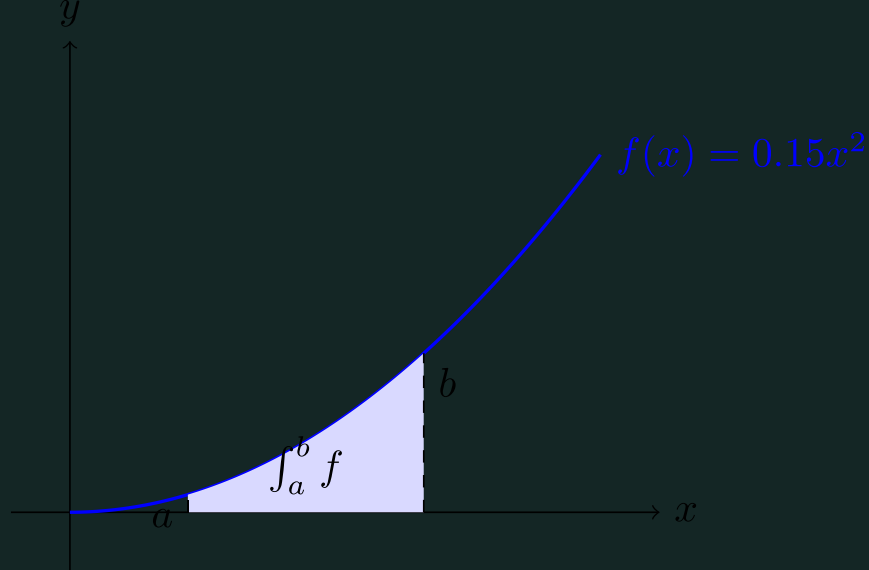
```

Interactive Diagram Selection

```

1 DIAGRAMS = {
2     "Function Plot": "diagrams/simple_shapes.tex",
3     "Neural Network": "diagrams/neural_network.tex",
4     "Finite Automaton": "diagrams/finite_automaton.tex",
5 }
6 PREAMBLES = {"Finite Automaton": r"\usetikzlibrary{automata,positioning}"}
7
8 choice = st.selectbox("Choose a TikZ diagram", [*DIAGRAMS])
9 stx.st_tikz(file=DIAGRAMS[choice], height=800, preamble=PREAMBLES.get(choice, ""))

```



Source: diagrams/simple_shapes.tex

```

1 \begin{tikzpicture}
2   % Axes
3   \draw[>-] (-0.5,0) -- (5,0) node[right] {$x$};
4   \draw[>-] (0,-0.5) -- (0,4) node[above] {$y$};
5
6   % Curve
7   \draw[blue, thick, domain=0:4.5, samples=100]
8     plot (\x, {0.15*\x*\x}) node[right] {$f(x) = 0.15x^2$};
9
10  % Filled area
11  \fill[blue!15, domain=1:3, samples=50]
12    (1,0) -- plot (\x, {0.15*\x*\x}) -- (3,0) -- cycle;
13
14  % Bounds
15  \draw[dashed] (1,0) -- (1,0.15) node[below left] {$a$};
16  \draw[dashed] (3,0) -- (3,1.35) node[below right] {$b$};
17
18  % Label
19
20

```

```
\node at (2,0.4) {\int_a^b f};  
\end{tikzpicture}
```

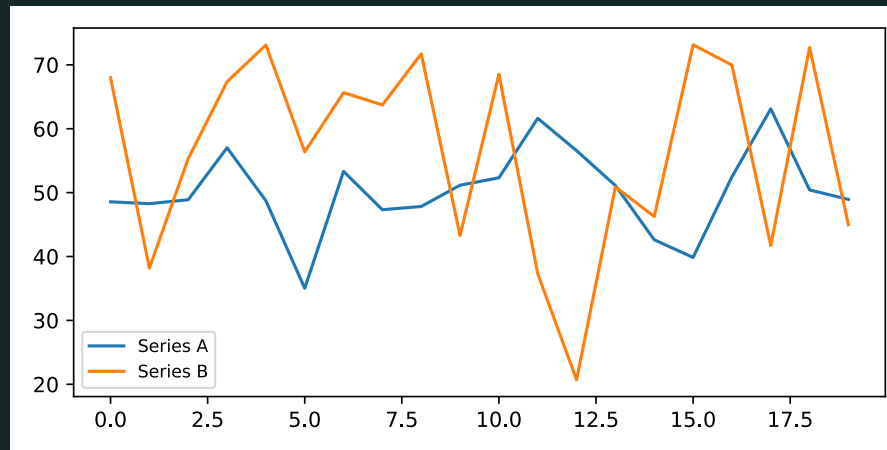
Details:

Built-in Charts

Purpose

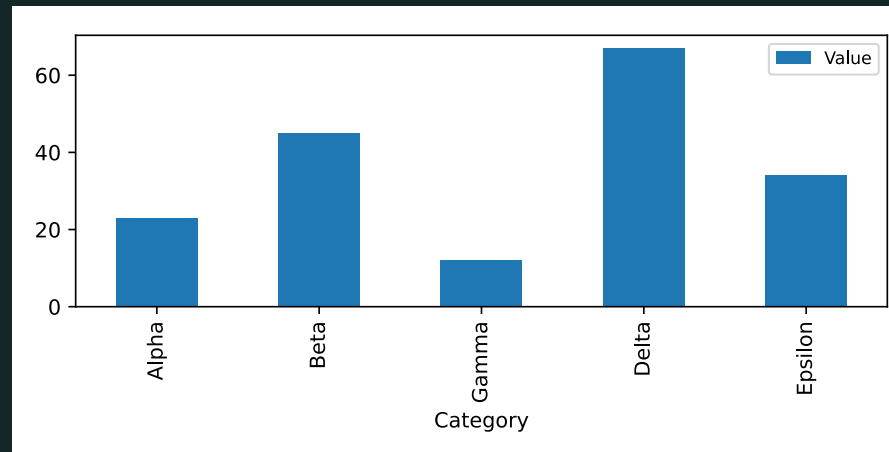
Line Chart

```
1 data = _make_line_data()  
2 with st_block(s.project.containers.result_box):  
3     stx.st_line_chart(data)
```



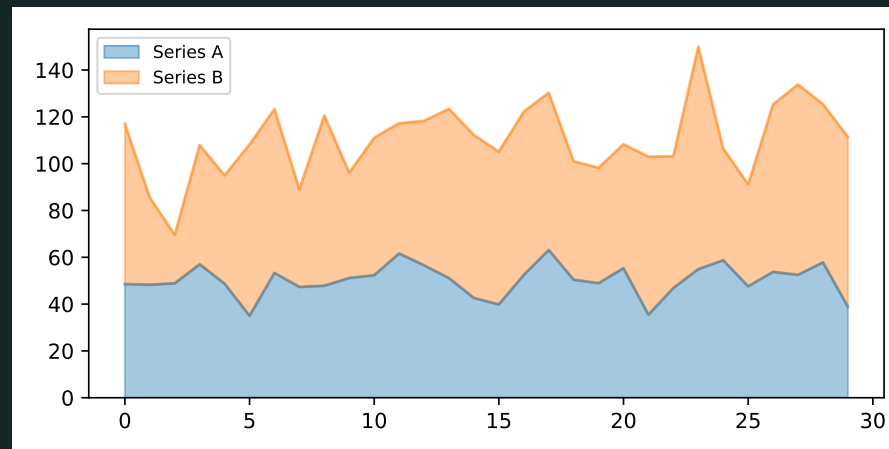
Bar Chart

```
1 bar_data = _make_bar_data()  
2 with st_block(s.project.containers.result_box):  
3     stx.st_bar_chart(bar_data, x="Category", y="Value")
```



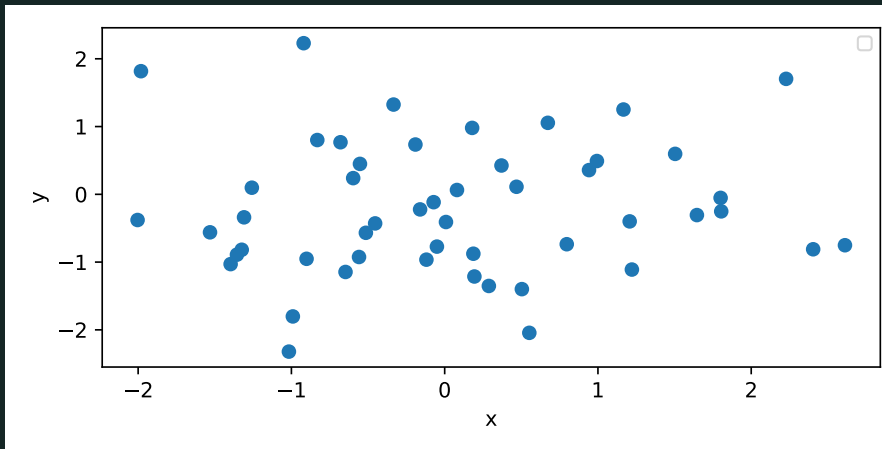
Interactive Area Chart

```
1 n_points = st.slider("Number of data points", 5, 100, 30,  
2                       key="bck27_area_slider")  
3 with st_block(s.project.containers.result_box):  
4     stx.st_area_chart(_make_line_data(n_points))
```



Scatter Chart

```
1 random.seed(0)
2 scatter_data = {
3     "x": [random.gauss(0, 1) for _ in range(50)],
4     "y": [random.gauss(0, 1) for _ in range(50)],
5     "size": [random.uniform(10, 60) for _ in range(50)],
6 }
7 with st_block(s.project.containers.result_box):
8     stx.st_scatter_chart(scatter_data, x="x", y="y", size="size")
```



Metrics & Chart in Grid

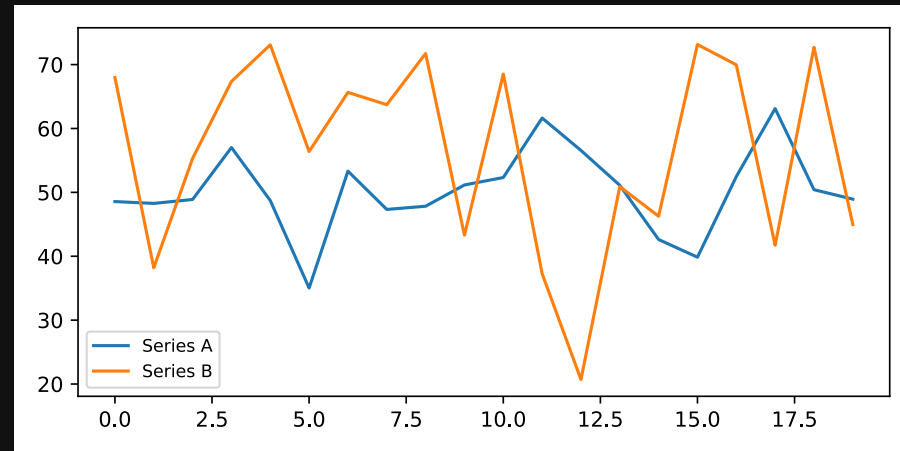
```
1 with st_grid(cols="1fr 2fr", cell_styles=bs.cell) as g:
2     with g.cell():
3         stx.st_metric("Users", "1,234", "+12%")
4         stx.st_metric("Revenue", "$5.6K", "+8%")
5         stx.st_metric("Uptime", "99.9%", "+0.1%")
```

```
6 with g.cell():
7     stx.st_line_chart(_make_line_data())
```

Users
1,234
+12%

Revenue
\$5.6K
+8%

Uptime
99.9%
+0.1%



Details:

DataFrames & Structured Data

Purpose

st.dataframe (interactive)

```
1 data = {
2     "Name": ["Alice", "Bob", "Charlie", "Diana"],
3     "Role": ["Engineer", "Designer", "Manager", "Analyst"],
4     "Score": [92, 87, 95, 88],
```

```
5 }  
6 stx.st_dataframe(data, use_container_width=True)
```

Name	Role	Score
Alice	Engineer	92
Bob	Designer	87
Charlie	Manager	95
Diana	Analyst	88

st.table (static)

```
1 data = {  
2     "Name": ["Alice", "Bob", "Charlie", "Diana"],  
3     "Role": ["Engineer", "Designer", "Manager", "Analyst"],  
4     "Score": [92, 87, 95, 88],  
5 }  
6 stx.st_table(data)
```

Name	Role	Score
Alice	Engineer	92
Bob	Designer	87
Charlie	Manager	95
Diana	Analyst	88

st.json (JSON viewer)

```
1 data = {  
2     "project": "StreamTeX",  
3     "version": "0.2.0",  
4     "features": ["styles", "grids", "lists", "export"],
```

```
5 }
6 stx.st_json(data)
```

```
{
  "project": "StreamTeX",
  "version": "0.2.0",
  "features": {
    "styles": true,
    "grids": true,
    "lists": true,
    "export": true
  }
}
```

StreamTeX Styled Grid

```
1 header_style = sg.create("A1:C1", bs.header_cell)
2 data_style = sg.create("A2:C5", bs.data_cell)
3 with st_grid(cols=3, cell_styles=header_style + data_style) as g:
4     for col in ["Name", "Role", "Score"]:
5         with g.cell(): st_write(col)
6     for i in range(len(data["Name"])):
7         for col in ["Name", "Role", "Score"]:
8             with g.cell(): st_write(str(data[col][i]))
```

Name	Role	Score
Alice	Engineer	92
Bob	Designer	87
Charlie	Manager	95
Diana	Analyst	88

Side-by-Side Comparison

Purpose

st.dataframe

Name	Role	Score
Alice	Engineer	92
Bob	Designer	87
Charlie	Manager	95
Diana	Analyst	88

st_grid

Name	Role	Score
Alice	Engineer	92
Bob	Designer	87
Charlie	Manager	95
Diana	Analyst	88

Details:

Google Sheets Data Import

Purpose

1. Configuration

```
1 from streamtex import GSheetConfig, AuthMode, set_gsheet_config
2
3 # Public sheets (no authentication needed)
4 cfg = GSheetConfig(auth_mode=AuthMode.PUBLIC, cache_ttl=600)
5
6 # Service account (server-side, recommended for deploy)
7 cfg = GSheetConfig(
8     auth_mode=AuthMode.SERVICE_ACCOUNT,
9     credentials_path="credentials.json", # or use env var
10    cache_ttl=300,
11 )
12
13 # OAuth2 fallback (interactive browser auth)
14 cfg = GSheetConfig(
15     auth_mode=AuthMode.OAUTH2,
16     credentials_path="client_secret.json",
17 )
18
19 set_gsheet_config(cfg)
```

2. Loading Data

```
1  from streamtex import GSheetSource, load_gsheet, load_gsheet_df
2
3  # From sheet ID
4  src = GSheetSource(sheet_id="1BxiMk...", tab="Notes", range="A1:E30")
5
6  # From URL
7  src = GSheetSource.from_url(
8      "https://docs.google.com/spreadsheets/d/1BxiMk.../edit",
9      tab="Notes",
10 )
11
12 # Load as list of dicts
13 data = load_gsheet(src)
14
15 # Load as pandas DataFrame
16 df = load_gsheet_df(src)
```

3. Live Demo

Using static fallback data (sheet unreachable)

stx.st_dataframe()

Student	Course	Grade	Semester
Alice	AI Fundamentals	92	S1
Bob	AI Fundamentals	87	S1
Charlie	Deep Learning	95	S1
Diana	Deep Learning	88	S1
Eve	NLP	91	S2
Frank	NLP	84	S2

Computed metrics

Students

6

Average Grade

89.5

Top Grade

95

4. Pattern: Data in Blocks

```
1 # blocks/_atomic/bck_grades.py
2 from streamtex import *
3 from streamtex import GSheetSource, load_gsheet_df, GSheetConfig, AuthMode
4 import streamtex as stx
5
6 _src = GSheetSource.from_url(
7     "https://docs.google.com/spreadsheets/d/1BxiMk.../edit",
8     tab="Notes",
9 )
10
11 def build():
12     df = load_gsheet_df(_src, config=GSheetConfig(auth_mode=AuthMode.PUBLIC))
13
14
```

```
stx.st_dataframe(df)
stx.st_bar_chart(df, x="Student", y="Grade")
```

Details:

5. Authentication Modes (AuthMode)

Purpose

```
1 from streamtex import GSheetConfig, AuthMode
2
3 # PUBLIC – no auth required (sheet must be "Anyone with link")
4 cfg_public = GSheetConfig(auth_mode=AuthMode.PUBLIC)
5
6 # SERVICE_ACCOUNT – server-side auth (recommended for production)
7 # Uses a JSON key file from Google Cloud Console
8 cfg_sa = GSheetConfig(
9     auth_mode=AuthMode.SERVICE_ACCOUNT,
10     credentials_path="path/to/service-account.json",
11 )
12
13 # OAUTH2 – interactive browser-based auth (for development)
14 cfg_oauth = GSheetConfig(auth_mode=AuthMode.OAUTH2)
```

Mode	Best For	Credentials
------	----------	-------------

PUBLIC	Public sheets, demos, tutorials	None required
SERVICE_ACCOUNT	Production apps, CI/CD, automated pipelines	JSON key from GCP Console
OAUTH2	Development, ad-hoc data access	OAuth2 client ID (browser auth flow)

Details:

Bibliography & References

Purpose

1. Loading References

```
1 # In book.py – load references at startup
2 from streamtex.bib import BibConfig, BibFormat, CitationStyle
3 from pathlib import Path
4
5 st_book(
6     module_list,
7     bib_sources=[str(Path(__file__).parent / "static" / "references.bib")],
8     bib_config=BibConfig(
9         format=BibFormat.APA,
10        citation_style=CitationStyle.AUTHOR_YEAR,
11        hover_enabled=True,
12        hover_show_abstract=True,
13    ),
14 )
```

Details:

2. Inline Citations with cite() and st_refs

```
1 from streamtex.bib import cite
2 from custom.bib_refs import st_refs # Generated - IDE completion!
3
4 # Option A: cite() with string keys
5 st_write(s.big,
6         "The Transformer architecture ",
7         cite("vaswani2017attention"),
8         " revolutionized NLP."
9     )
10
11 # Option B: st_refs with IDE autocompletion (recommended)
12 st_write(s.big,
13         "The Transformer architecture ",
14         st_refs.vaswani2017attention, # ← Ctrl+Space completes!
15         " revolutionized NLP."
16     )
17
18 # For multi-key citations, use cite() directly:
19 cite("lecun2015deep", "goodfellow2016deep") # (LeCun; Goodfellow)
```

Live result (using st_refs):

The Transformer architecture (Vaswani et al., 2017) revolutionized NLP. Pre-trained models like BERT (Devlin et al., 2019) extended this to transfer learning.

Scaling laws (Brown et al., 2020) showed that larger models achieve remarkable few-shot performance, building on foundational work in deep learning (LeCun et al., 2015; Goodfellow et al., 2016).

3. Citation Styles

```
1 from streamtex.bib import CitationStyle
2
3 # (Vaswani et al., 2017) – default
4 BibConfig(citation_style=CitationStyle.AUTHOR_YEAR)
5
6 # [1]
7 BibConfig(citation_style=CitationStyle.NUMERIC)
8
9 # ^1 (superscript)
10 BibConfig(citation_style=CitationStyle.SUPERSCRIPT)
```

4. Import Formats

Format	Extension	Source	Function
BibTeX	.bib	LaTeX / Google Scholar	load_bibtex()
JSON	.json	Custom / API	load_bib_json()
RIS	.ris		load_bib_ris()

		Zotero / Mendeley / EndNote	
CSL-JSON	.csl-json	Zotero native export	load_bib_csl_json()

```

1  # Auto-detect by extension
2  from streamtex.bib import load_bib
3  entries = load_bib("references.bib")    # BibTeX
4  entries = load_bib("refs.ris")         # RIS
5  entries = load_bib("refs.json")        # JSON
6
7  # Add a custom parser
8  from streamtex.bib import register_bib_parser
9
10 def parse_yaml(path):
11     import yaml
12     with open(path) as f:
13         data = yaml.safe_load(f)
14     return [BibEntry(key=d["key"], title=d["title"]) for d in data]
15
16 register_bib_parser("yaml", parse_yaml)

```

5. Extensible Fields

Purpose

```

1 entry = BibEntry(
2     key="example",
3     title="My Paper",
4     authors=["Smith, J."],
5     year="2024",
6     # Standard fields
7     isbn="978-3-16-148410-0",
8     keywords="AI, deep learning",
9     institution="MIT",
10    # Extra fields preserved automatically from BibTeX
11    extra={"funding": "NSF Grant 12345"},
12 )
13
14 entry.get_field("isbn")          # "978-3-16-148410-0"
15 entry.get_field("funding")      # "NSF Grant 12345" (from extra)
16 entry.get_field("missing", "N/A") # "N/A"

```

6. Registry Status

Registered

5

Cited

5

Available keys

5

Registered entries:

vaswani2017attention (article) *Attention Is All You Need*
devlin2019bert (inproceedings) *BERT: Pre-training of Deep Bidirectional Transformers for La...*
brown2020language (article) *Language Models are Few-Shot Learners*
goodfellow2016deep (book) *Deep Learning*

7. Output Formats (BibFormat)

Purpose

```
1 from streamtex.bib import BibFormat
2
3 # APA (default) — Author, A. B. (Year). Title. Journal, Vol(Num), Pages.
4 BibConfig(format=BibFormat.APA)
5
6 # MLA — Author. "Title." Journal, vol. Vol, no. Num, Year, pp. Pages.
7 BibConfig(format=BibFormat.MLA)
8
9 # IEEE — [1] A. Author, "Title," Journal, vol. Vol, no. Num, pp. Pages, Year.
10 BibConfig(format=BibFormat.IEEE)
11
12 # Chicago — Author. "Title." Journal Vol, no. Num (Year): Pages.
13 BibConfig(format=BibFormat.CHICAGO)
14
15 # Harvard — Author Year, 'Title', Journal, vol. Vol, no. Num, pp. Pages.
16 BibConfig(format=BibFormat.HARVARD)
```

Details:

8. Rendered Bibliography (APA)

```
1 from streamtex.bib import st_bibliography
2
3 st_bibliography(
4     title="References",
5     toc_lvl="1",
6     only_cited=True, # Only show entries cited via cite()
7     format=BibFormat.APA,
8 )
```

[1] Brown, Tom B., Mann, Benjamin, Ryder, Nick, Subbiah, Melanie, Kaplan, Jared, Dhariwal, Prafulla, Neelakantan, Arvind, Shyam, Pranav, Sastry, Girish, & Askell, Amanda (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877--1901. <https://doi.org/10.48550/arXiv.2005.14165>

[2] Devlin, Jacob, Chang, Ming-Wei, Lee, Kenton, & Toutanova, Kristina (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*. <https://doi.org/10.18653/v1/N19-1423>

[3] Goodfellow, Ian, Bengio, Yoshua, & Courville, Aaron (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org>

[4] LeCun, Yann, Bengio, Yoshua, & Hinton, Geoffrey (2015). Deep Learning. *Nature*, 521(7553), 436--444. <https://doi.org/10.1038/nature14539>

[5] Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, & Polosukhin, Illia (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30, 5998--6008. <https://doi.org/10.48550/arXiv.1706.03762>

9. BibTeX Export

```
1 from streamtex.bib import export_bibtex
2
3 bibtex_str = export_bibtex(only_cited=True)
4 st.download_button("Download .bib", bibtex_str, "references.bib")
```

10. st_refs & IDE Autocompletion

Purpose

```
1 # Step 1: Generate typed module from your bibliography file
2 # Run once (or whenever your .bib changes):
3 #   uv run python -m streamtex generate-stubs static/refs.bib -o custom/bib_refs.py
4
5 # Step 2: Import st_refs from the generated module
6 from custom.bib_refs import st_refs # ← IDE sees all keys!
7
8 st_write(s.big,
9     "According to ", st_refs.vaswani2017attention, # Ctrl+Space!
10     " the Transformer architecture..."
11 )
12
13 # cite() still works for multi-key and prefix/suffix:
14 cite("lecun2015deep", "goodfellow2016deep") # (LeCun; Goodfellow)
15 cite("vaswani2017attention", prefix="cf. ", suffix=", p. 42")
```

Details:

11. Custom Parsers (register_bib_parser)

Purpose

```
1 from streamtex.bib import register_bib_parser, BibEntry
2
3 def parse_custom_format(filepath: str) -> list[BibEntry]:
4     """Parse a custom bibliography format."""
5     entries = []
6     with open(filepath) as f:
7         for line in f:
8             # Parse your custom format here
9             key, title, author, year = line.strip().split("|")
10            entries.append(BibEntry(
11                key=key, title=title, authors=[author], year=year
12            ))
13    return entries
14
15 # Register for .custom extension
16 register_bib_parser("custom", parse_custom_format)
17
18 # Now load_bib() auto-detects .custom files
19 # load_bib("references.custom")
```

Details:

12. set_bib_config() — Global Configuration

Purpose

```
1 from streamtex.bib import BibConfig, BibFormat, CitationStyle, set_bib_config
2
3 # Configure bibliography globally
4 set_bib_config(BibConfig(
5     format=BibFormat.IEEE,
6     citation_style=CitationStyle.NUMERIC,
7     hover_enabled=True,
8     hover_show_abstract=True,
9     sort_by="year",          # "author" | "year" | "key" | "citation_order"
10    locale="en",             # "en" or "fr"
11 ))
```

Details:

13. format_entry() — Custom Entry Formatting

Purpose

```
1 from streamtex.bib import format_entry, BibFormat, get_bib_registry
2
3 reg = get_bib_registry()
4 entry = reg.get("vaswani2017attention")
5
6 # Format as APA
7 apa_html = format_entry(entry, BibFormat.APA)
8
9 # Format as IEEE with a citation number
```

```
10 ieee_html = format_entry(entry, BibFormat.IEEE, number=1)
11
12 # Use in st_write via raw HTML
13 st_write(s.medium, apa_html)
```

Details:

Interactive Widgets

Purpose

Buttons & Toggles

```
1 if st.button("Say hello", key="bck25_hello"):
2     with st_block(s.project.containers.result_box):
3         st_write(s.large + s.bold + s.project.colors.success_green,
4                 "Hello from StreamTeX!")
5 show = st.toggle("Show styled block", key="bck25_toggle")
6 if show:
7     with st_block(s.project.containers.tip_callout):
8         st_write(s.large, "This block appears when toggled on.")
```

Radio & Selectbox

```
1 choice = st.radio("Pick a callout style",
2                   ["Good", "Bad", "Tip", "Note"],
3                   key="bck25_radio", horizontal=True)
4 callout_map = {
5     "Good": s.project.containers.good_callout,
6     "Bad": s.project.containers.bad_callout,
7     "Tip": s.project.containers.tip_callout,
8     "Note": s.project.containers.note_callout,
9 }
```

```
10 with st_block(callout_map[choice]):
11     st_write(s.large, f"This is a {choice} callout.")
```

This is a Good callout.

Slider-Driven Styles

```
1 size = st.slider("Font size (px)", 12, 96, 32, key="bck25_slider")
2 dynamic_style = ns(f"font-size: {size}px;")
3 with st_block(s.project.containers.result_box):
4     st_write(dynamic_style + s.bold, "Dynamic size text")
```

Dynamic size text

Text Input Live Preview

```
1 user_text = st.text_input("Type something", key="bck25_text",
2                             value="Hello StreamTeX")
3 if user_text:
4     with st_block(s.project.containers.result_box):
5         st_write(s.large + s.italic, user_text)
6         st_br()
7         st_write(s.large + s.bold + s.project.colors.primary_blue,
8                 user_text)
9         st_br()
10        st_write(s.large + s.text.decorator underline_text
11                + s.project.colors.accent_teal, user_text)
```

Hello StreamTeX

Hello StreamTeX

Hello StreamTeX

Color Picker

```
1 picked = st.color_picker("Pick a text color", "#E74C3C",  
2                             key="bck25_color")  
3 color_style = ns(f"color: {picked};")  
4 with st_block(s.project.containers.result_box):  
5     st_write(color_style + s.Large + s.bold, "Colored text!")
```

Colored text!

Details:

Forms & Session State

Purpose

Basic Form

```
1 with st.form("bck26_profile_form"):
2     name = st.text_input("Name", value="Ada Lovelace")
3     age = st.number_input("Age", 0, 120, 25)
4     submitted = st.form_submit_button("Submit")
5 if submitted:
6     with st_block(s.project.containers.result_box):
7         st_write(s.large + s.bold, f"Name: {name}")
8         st_br()
9         st_write(s.large, f"Age: {age}")
```

Session State Counter

```
1 if "bck26_counter" not in st.session_state:
2     st.session_state.bck26_counter = 0
3 if st.button("Increment", key="bck26_incr"):
4     st.session_state.bck26_counter += 1
5 count = st.session_state.bck26_counter
6 size = min(12 + count * 4, 96)
7 with st_block(s.project.containers.result_box):
8     st_write(ns(f"font-size: {size}px;") + s.bold,
9             f"Count: {count}")
```

Count: 0

Form with Styled Feedback

```
1 with st.form("bck26_validation_form"):
2     password = st.text_input("Enter a password", type="password")
3     validated = st.form_submit_button("Validate")
4     if validated:
5         if len(password) >= 8:
6             with st_block(s.project.containers.good_callout):
7                 st_write(s.project.titles.tip_label, "Valid")
8                 st_space("v", 1)
9                 st_write(s.large, "Strong password!")
10        else:
11            with st_block(s.project.containers.bad_callout):
12                st_write(s.project.titles.warning_label, "Invalid")
13                st_space("v", 1)
14                st_write(s.large,
15                        f"Too short ({len(password)}/8 characters).")
```

Persistent List

```
1  if "bck26_items" not in st.session_state:
2      st.session_state.bck26_items = []
3  with st.form("bck26_add_item_form"):
4      new_item = st.text_input("New item")
5      added = st.form_submit_button("Add to list")
6  if added and new_item:
7      st.session_state.bck26_items.append(new_item)
8  if st.session_state.bck26_items:
9      with st_block(s.project.containers.result_box):
10         with st_list(list_type=lt.ordered, li_style=s.large) as l:
11             for item in st.session_state.bck26_items:
12                 with l.item():
13                     st_write(item)
```

Details:

The Problem: `st.*` Widgets Disappear in Export

⚠ Native Streamlit widgets are invisible in HTML export

When you enable HTML export (`st_book(..., export=True)`), native `st.*` widgets (`st.dataframe`, `st.line_chart`, `st.table`, etc.) become invisible because they use Streamlit's React pipeline which doesn't export to static HTML.

The Solution: Use `stx.st_*` Wrappers

✓ Use `stx.st_*` functions instead

StreamTeX provides export-aware wrappers (`stx.st_dataframe`, `stx.st_line_chart`, etc.) that render BOTH in the live app AND in exported HTML with static fallbacks.

Available Export-Aware Widgets

- `stx.st_dataframe()` — tables
- `stx.st_line_chart()` — line charts
- `stx.st_bar_chart()` — bar charts
- `stx.st_area_chart()` — area charts
- `stx.st_scatter_chart()` — scatter plots
- `stx.st_table()` — simple tables
- `stx.st_metric()` — metrics
- `stx.st_json()` — JSON display
- `stx.st_graphviz()` — graph diagrams
- `stx.st_audio()` — audio playback
- `stx.st_video()` — video playback

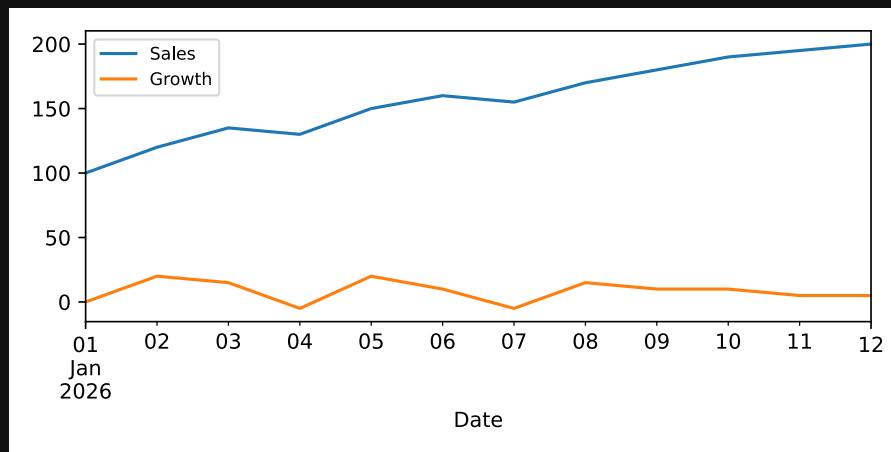
Live Demo: Dataframe

This table will appear in both the live app AND the exported HTML:

Name	Score	Status
Alice	95	Pass
Bob	87	Pass
Charlie	92	Pass
Diana	88	Pass

Live Demo: Line Chart

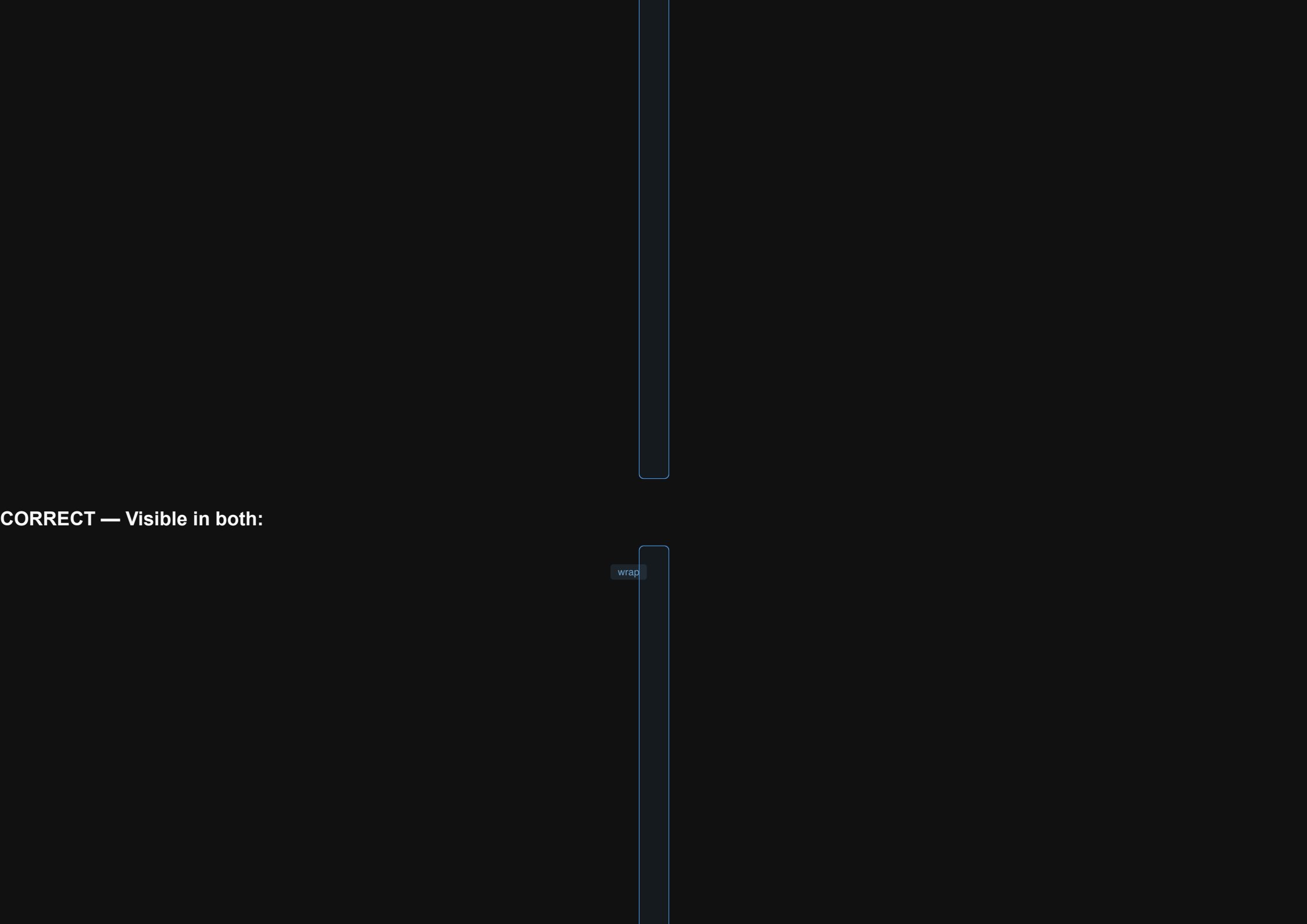
This chart will appear in both the live app AND the exported HTML:



Code Examples

WRONG — Won't appear in export:

wrap



CORRECT — Visible in both:

wrap

st_graphviz — SVG Export

`stx.st_graphviz()` renders Graphviz diagrams as inline SVG in exports, making them resolution-independent and searchable:

wrap

Best Practices

1. Always use `stx.st_*` for widgets when export is enabled
2. Use `st.*` ONLY for interactive elements (buttons, inputs) that won't export
3. For custom widgets without `stx.st_*` wrapper, use `stx.st_export()` context
4. Test HTML export to ensure all data is visible

Purpose

ExportConfig

Purpose

```
1  from streamtex import ExportConfig
2
3  export_cfg = ExportConfig(
4      enabled=True,          # enable HTML export
5      page_title="My Export", # <title> of exported HTML
6      page_width="90%",      # max-width of the page container
7      page_padding="20px",   # padding around the page
8  )
9
10 st_book(blocks, export=export_cfg)
```

- **enabled** — toggle export on or off (default False)
- **page_title** — the HTML page title in the exported file
- **page_width** — CSS max-width for the page container (default 90%)
- **page_padding** — CSS padding around the main content area

How the Export Buffer Works

Purpose

```
1 # Simplified internal flow:
2 #
3 # 1. st_book() calls reset() on the buffer
4 # 2. Each block's build() runs:
5 #     - st_write() appends <div>...</div> to buffer
6 #     - st_code() appends <pre><code>...</code></pre> to buffer
7 #     - st_block() pushes/pops wrapper divs
8 # 3. After all blocks: generate_full_html() produces the file
```

The Push/Pop Wrapper Pattern

Purpose

```
1 # Internal mechanism (you don't call this directly):
2 export_push_wrapper('<div style="display:flex; gap:16px;">')
3 # ... content rendered inside appends to the buffer ...
4 export_pop_wrapper('</div>')
5
6 # This happens automatically when you use context managers:
7 with st_block(my_style):
8     st_write(s.medium, "Content inside the block")
9     # push on __enter__, pop on __exit__
```

st_html() — The Dual-Render Bridge

Purpose

```
1 from streamtex import st_html
2
3 # Inline mode (height=0): renders via st.html()
4 st_html(html_string, height=0)
5
6 # Iframe mode (height>0): renders via components.html()
7 st_html(html_string, height=400, light_bg=True, scrolling=True)
```

- **height=0** — inline rendering via `st.html()` (no iframe, content flows with page)
- **height>0** — iframe rendering via `components.html()` (isolated, fixed height in px)
- **light_bg** — injects color-scheme: light for white backgrounds inside the iframe
- **scrolling** — enables scrollbar in iframe mode (default False)

Export-Aware Widgets Reminder

Important: native `st.*` widgets (`st.dataframe`, `st.line_chart`, etc.) are invisible in exported HTML. Always use their `stx.st_*` equivalents when export is enabled.

```
1 # WRONG - disappears in export
2 st.dataframe(df)
3
4 # CORRECT - visible in both live app and exported HTML
5 stx.st_dataframe(df)
```

See the **Export-Aware Widgets** section for the full list of available `stx.st_*` wrappers.

Full Export Lifecycle

The complete export cycle follows these steps:

1. `st_book()` detects export is enabled and calls `buffer.reset()`
2. The book renders all blocks in order (paginated or continuous)
3. Each stx function appends its HTML fragment to the buffer
4. Nested containers use push/pop to maintain proper HTML structure
5. After all blocks render, `generate_full_html()` assembles the output
6. CSS from `default.css` is inlined into the HTML document
7. The final HTML file is offered for download via Streamlit

CSS Bundling

Purpose

```
1  # Simplified generate_full_html() output structure:
2  #
3  # <!DOCTYPE html>
4  # <html>
5  # <head>
6  #   <title>{page_title}</title>
7  #   <style>
8  #     /* contents of default.css inlined here */
9  #   </style>
10 # </head>
11 # <body>
12 #   <div style="max-width:{page_width}; padding:{page_padding};">
13 #     {accumulated_html_fragments}
14 #   </div>
15 #
16
```

```
# </body>
# </html>
```

Details:

PDF Export

Purpose

1. PdfMode

Purpose

```
1  from streamtex import PdfMode
2
3  # Continuous mode – single flowing document
4  mode = PdfMode.CONTINUOUS
5
6  # Paginated mode – page break at each slide break
7  mode = PdfMode.PAGINATED
```

[wrap](#)

2. PdfConfig

Purpose

```
1  from streamtex import PdfConfig, PdfMode
2
3  config = PdfConfig(
```

[wrap](#)

```

4     mode=PdfMode.PAGINATED,
5     format="A4",           # A4, Letter, A3, Legal, Tabloid
6     landscape=True,       # landscape orientation
7     margin_top="10mm",    # CSS margin values
8     margin_bottom="10mm",
9     margin_left="15mm",
10    margin_right="15mm",
11    print_background=True, # include background colors
12    scale=1.0,             # scale factor (0.1 - 2.0)
13    page_numbers=True,     # add page numbers in footer
14 )

```

- **mode** — PdfMode.CONTINUOUS or PdfMode.PAGINATED
- **format** — page format string (A4, Letter, etc.)
- **landscape** — landscape orientation (default True)
- **scale** — scale factor from 0.1 to 2.0 (default 1.0)
- **page_numbers** — add centered page numbers in footer
- **header_template / footer_template** — custom Chromium print header/footer HTML

3. Usage in st_book()

Purpose

```

1  from streamtex import st_book, PdfConfig, PdfMode
2
3  st_book(
4      [bck_intro, bck_chapter_1, bck_chapter_2],
5      pdf_config=PdfConfig(
6          mode=PdfMode.PAGINATED,

```

```
7         format="A4",
8         landscape=True,
9         margin_top="0",
10        margin_bottom="0",
11        margin_left="0",
12        margin_right="0",
13        page_numbers=True,
14    ),
15 )
```

4. export_pdf() — Programmatic Export

Purpose

```
1  from streamtex import export_pdf, PdfConfig, PdfMode
2  from streamtex.export import generate_export_html
3
4  # Get the HTML from the export buffer
5  html = generate_export_html()
6
7  # Export to PDF bytes
8  pdf_bytes = export_pdf(html, config=PdfConfig(
9      mode=PdfMode.PAGINATED,
10     format="A4",
11 ))
12
13 # Or export directly to a file
14 export_pdf(html, output_path="output.pdf", config=PdfConfig(
15     mode=PdfMode.CONTINUOUS,
16 ))
17
```

```
scale=0.8,  
))
```

Details:

5. Auto-Export to Disk

Purpose

```
1  from streamtex import st_book, ExportConfig, ExportMode, PdfConfig  
2  
3  st_book([...],  
4          exports=[  
5              # HTML with timestamp: my-course-260318-155306.html  
6              ExportConfig(  
7                  format="html",  
8                  mode=ExportMode.ALWAYS,  
9                  output_dir="./exports",  
10                 filename="my-course",  
11                 timestamp=True,  
12             ),  
13             # PDF for printing (A4 portrait)  
14             ExportConfig(  
15                 format="pdf",  
16                 mode=ExportMode.ALWAYS,  
17                 output_dir="./exports",  
18                 filename="my-course-print",  
19                 timestamp=True,  
20             )
```

```
21         pdf=PdfConfig(format="A4", landscape=False),
22     ),
23     # PDF for projection (A4 landscape, no margins)
24     ExportConfig(
25         format="pdf",
26         mode=ExportMode.ALWAYS,
27         output_dir="./exports",
28         filename="my-course-slides",
29         timestamp=False,
30         pdf=PdfConfig(
31             format="A4", landscape=True,
32             margin_top="0", margin_bottom="0",
33             margin_left="0", margin_right="0",
34         ),
35     ),
36 ],
37 )
```

Details:

Static Assets

Folder organization

Purpose

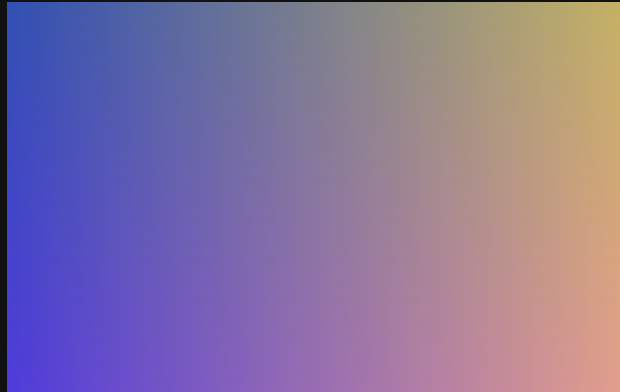
```
1 # .streamlit/config.toml
2 [server]
3 enableStaticServing = true
4
5 # Folder structure
6 project_name/
7   static/
8     images/      # PNG, JPG, SVG, ...
9     videos/      # MP4, WebM, ...
10    texts/       # TXT, Markdown, ...
11    pdf/         # PDF documents
12    sounds/      # WAV, MP3, ...
13    various/     # JSON, CSV, ...
```

Local images

Purpose

```
1 configure_image_path("app/static/images")
2 st_image(uri="sample_gradient.png",
```

```
3 width="400px", height="250px",  
4 alt="Sample gradient image")
```



Text files

Purpose

```
1 text_path = os.path.join("static", "texts", "sample_lorem.txt")  
2 with open(text_path) as f:  
3     content = f.read()  
4 stx.st_code(code=content, language="text")
```

```
1 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut  
2 labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco  
3 laboris nisi ut aliquip ex ea commodo consequat.  
4  
5 Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla  
pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt  
mollit anim id est laborum.
```

StreamTeX est une bibliothèque puissante pour créer des applications web interactives avec Streamlit. Elle offre un système de styles flexible et modulaire.

PDF documents

Purpose

```
1 pdf_url = "app/static/pdf/sample_document.pdf"
2 st_html(
3     f'<iframe src="{pdf_url}" '
4     f'width="100%" height="400" '
5     f'style="border:none;"></iframe>'
6 )
```

 PDF preview not available (sample_document.pdf not found)

Audio files

Purpose

```
1 audio_path = os.path.join("static", "sounds", "sample_tone.wav")
2 stx.st_audio(audio_path, format="audio/wav")
```

▶ 0:00 / 0:02 

Local video files

Purpose

```
1 video_path = os.path.join("static", "videos", "chameleon.mp4")
2 stx.st_video(video_path)
```

wrap

▶ 0:00



YouTube videos

Purpose

```
1 # Read URL from a file
2 url_file = os.path.join("static", "videos", "youtube_urls.txt")
3 with open(url_file) as f:
4     youtube_url = f.readline().strip()
5 stx.st_video(youtube_url)
```

wrap

Data files (JSON, CSV, ...)

Purpose

```
1 import json
2 json_path = os.path.join("static", "various", "sample_data.json")
3 with open(json_path) as f:
4     data = json.load(f)
5 stx.st_code(code=json.dumps(data, indent=2), language="json")
```

```
1 {
2     "name": "StreamTeX Project",
3     "version": "0.2.0",
4     "features": [
5         "Modular styling system",
6         "Lazy-loading blocks",
7         "Static asset resolution",
8         "HTML export"
9     ]
10 }
```

```
10 ],
11 "stats": {
12   "blocks": 32,
13   "tests": 203,
14   "components": 15
15 }
```

Details:

Deploy with Docker

The StreamTeX Dockerfile

Purpose

```
1 # Dockerfile not found at expected path
```

Build and run locally

Purpose

```
1 # Build the image
2 docker build -t streamtex-app .
3
4 # Run the container
5 docker run -p 8501:8501 streamtex-app
```

Targeting a specific project

Purpose

```
1 # Deploy a specific project
2 docker build \
3     --build-arg FOLDER=projects/my_project \
4     -t my-project-app .
5
6 docker run -p 8501:8501 my-project-app
```

Environment variables

Purpose

Variable	Purpose
STREAMLIT_SERVER_HEADLESS	Run without opening a browser
PYTHONDONTWRITEBYTECODE	Skip .pyc file creation
UV_LINK_MODE=copy	Copy files instead of hardlinks (Docker compat)

Details:

Deploy on Hugging Face Spaces

What is Hugging Face Spaces?

Purpose

Create a Space

Purpose

```
1 # Steps:
2 # 1. Go to https://huggingface.co/new-space
3 # 2. Name your Space (e.g. "my-streamtex-app")
4 # 3. Select "Docker" as the Space SDK
5 # 4. Choose visibility (public or private)
6 # 5. Click "Create Space"
```

Push your project

Purpose

```
1 # Add HF remote
2 git remote add hf https://huggingface.co/spaces/USERNAME/SPACE_NAME
3
4 # Push to Hugging Face
5 git push hf main
```

Expected repository structure

Purpose

```
1 # Repository root
2 Dockerfile           # <- HF builds this
3 pyproject.toml
4 uv.lock
5 streamtex/           # Library
6 projects/
7   my_project/
8     book.py
9     blocks/
10    custom/
```

README.md metadata

Purpose

```
1 ---
2 title: My StreamTeX App
3 emoji: 📄
4 colorFrom: blue
5 colorTo: purple
6 sdk: docker
7 app_port: 8501
8 ---
```

Details:

Deploy on Cloud (GCP / AWS)

General approach

Purpose

```
1 # General workflow
2 # 1. Build the Docker image locally
3 # 2. Tag it for your registry
4 # 3. Push to the registry
5 # 4. SSH into your VM
6 # 5. Pull and run the image
```

GCP: Google Compute Engine

Purpose

```
1 # Build and tag for GCP Artifact Registry
2 docker build -t streamtex-app .
3 docker tag streamtex-app \
4     REGION-docker.pkg.dev/PROJECT_ID/REPO/streamtex-app:latest
5
6 # Push to Artifact Registry
```

```
7 docker push \  
8     REGION-docker.pkg.dev/PROJECT_ID/REPO/streamtex-app:latest  
9  
10 # On the GCP VM (after SSH):  
11 docker pull \  
12     REGION-docker.pkg.dev/PROJECT_ID/REPO/streamtex-app:latest  
13 docker run -d -p 8501:8501 \  
14     REGION-docker.pkg.dev/PROJECT_ID/REPO/streamtex-app:latest
```

AWS: EC2 with ECR

Purpose

```
1 # Authenticate with ECR  
2 aws ecr get-login-password --region REGION | \  
3     docker login --username AWS --password-stdin \  
4     ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com  
5  
6 # Build, tag and push  
7 docker build -t streamtex-app .  
8 docker tag streamtex-app \  
9     ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/streamtex-app:latest  
10 docker push \  
11     ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/streamtex-app:latest  
12  
13 # On the EC2 instance (after SSH):  
14 docker pull \  
15     ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/streamtex-app:latest  
16 docker run -d -p 8501:8501 \  
17     ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/streamtex-app:latest
```

Essential Docker commands

Command	Purpose
<code>docker build -t name .</code>	Build image from Dockerfile
<code>docker tag src dst</code>	Tag image for a registry
<code>docker push image</code>	Upload image to registry
<code>docker pull image</code>	Download image from registry
<code>docker run -d -p 8501:8501</code>	Run container in background

Details:

Presentation Mode

Purpose

PresentationConfig

Purpose

```
1  from streamtex import PresentationConfig
2
3  config = PresentationConfig(
4      # Identity
5      title="Introduction to Docker",
6      subtitle="A hands-on workshop",
7
8      # Aspect ratio
9      aspect_ratio="16/9",          # CSS aspect-ratio ("16/9", "4/3", "16/10")
10     enforce_ratio=True,          # Apply aspect-ratio + overflow:hidden
11
12     # Footer
13     footer=True,                  # Show fixed slide counter bar
14     counter_mode="bloc",          # "bloc" = section count, "slide" = marker count
15     footer_height="48px",         # CSS height of the footer bar
16     footer_bg=None,              # Background colour (None = inherit theme)
17     footer_text_color=None,       # Text colour (None = inherit theme)
18     footer_font_size="18px",      # Font size for footer text
19
20
```

```

21 # Layout
22 center_content=True,      # Vertically centre slide content
23 content_padding="48px 64px", # CSS padding inside each slide
24
25 # Streamlit UI
26 hide_streamlit_header=True, # Hide the Streamlit header bar
27 hide_streamlit_footer=True, # Hide the "Made with Streamlit" footer
28 hide_deploy_button=True,    # Hide the deploy button
29 sidebar_default="collapsed", # Initial sidebar state
30
31 # Transitions (future)
32 slide_transition="none",    # "none", "fade", "slide"
33 transition_duration="0.3s",
34 )

```

Setup in book.py

Purpose

```

1 from streamtex import (
2     st_book, PresentationConfig, set_presentation_config,
3     add_presentation_options,
4     SlideBreakConfig, SlideBreakMode, set_slide_break_config,
5 )
6
7 # 1. Configure presentation mode
8 set_presentation_config(PresentationConfig(
9     title="My Presentation",
10    aspect_ratio="16/9",
11    footer=True,
12

```

```

13     center_content=True,
14     hide_streamlit_header=True,
15     enforce_ratio=True,
16 ))
17
18 # 2. Configure slide breaks
19 set_slide_break_config(SlideBreakConfig(
20     mode=SlideBreakMode.FULL,
21     space="5vh",
22     marker=True,
23 ))
24
25 # 3. Add sidebar controls (toggle footer, fullscreen)
26 add_presentation_options()
27
28 # 4. Launch the book
29 st_book([
30     blocks.bck_title_slide,
31     blocks.bck_agenda,
32     blocks.bck_content,
33     blocks.bck_conclusion,
34 ], paginate=True, view_modes=[ViewMode.PAGINATED, ViewMode.CONTINUOUS])

```

View Modes

Purpose

```

1  from streamtex import st_book, ViewMode
2
3  # Both modes available (default — radio button visible)

```

wrap

```
4 st_book(blocks, view_modes=[ViewMode.PAGINATED, ViewMode.CONTINUOUS])
5
6 # Paginated only – radio button hidden
7 st_book(blocks, view_modes=[ViewMode.PAGINATED])
8
9 # Continuous only – radio button hidden
10 st_book(blocks, view_modes=[ViewMode.CONTINUOUS])
11
12 # None = both modes (same as omitting the parameter)
13 st_book(blocks, view_modes=None)
```

Details:

Presentation Footer

Purpose

```
1 from streamtex import st_presentation_footer, PresentationConfig
2
3 # Automatic: st_book() renders the footer when PresentationConfig is set.
4 # No manual call needed in the standard workflow.
5
6 # Manual usage for custom setups:
7 st_presentation_footer(
8     current_slide=3,
9     total_slides=12,
10     title="My Presentation",      # Falls back to config.title if omitted
11
12
```

```
config=None, # Falls back to get_presentation_config()
)
```

Details:

Sidebar Controls

Purpose

```
1 from streamtex import add_presentation_options
2
3 # Add controls to the sidebar (default)
4 add_presentation_options()
5
6 # Add controls to a custom container
7 import streamlit as st
8 with st.expander("Presenter Controls"):
9     add_presentation_options(container=st)
```

Details:

Section Spacing with Profiles

Purpose

Per-Profile Spacing

```
1  from streamtex import (  
2      PresentationProfile, PageLayout, ViewMode,  
3      Spacing, SpacingConfig,  
4  )  
5  
6  presentation_profiles=[  
7      PresentationProfile(  
8          name="Slides",  
9          mode=ViewMode.PAGINATED,  
10         layout=PageLayout(width=100, zoom=100),  
11         spacing=SpacingConfig(  
12             block=Spacing(top=0),          # no gap in paginated  
13             section=Spacing(top=3),        # generous section spacing  
14         ),  
15     ),  
16     PresentationProfile(  
17         name="Handout",  
18         mode=ViewMode.CONTINUOUS,  
19         layout=PageLayout(width=80, zoom=90),  
20         spacing=SpacingConfig(  
21             block=Spacing(top="70px"),      # visible gap between blocks  
22             section=Spacing(top=1, left="5%", right="5%"),  
23         ),  
24     ),  
25 ]
```

```
),  
]
```

Override Hierarchy

Purpose

Double-Spacing Prevention

Purpose

Horizontal Margins

Purpose

```
1  from streamtex import st_slide_break, Spacing  
2  
3  # Indented section (both text and containers move)  
4  st_slide_break("Narrow Section", spacing=Spacing(  
5      top=2,  
6      left="15%",  
7      right="15%",  
8  ))  
9  
10 # Asymmetric indent (shifted right)  
11 st_slide_break("Right-shifted", spacing=Spacing(  
12     top=2,  
13     left="15%",  
14     right="15%",  
15 ))
```

wrap

Changelog